Lab 3.1 : Exposer une application

*Vérification de nodes :

```
arij@master:~$ kubectl get nodes -o wide
NAME      STATUS   ROLES           AGE   VERSION        INTERNAL-IP      EXTERNAL-IP   OS-IMAGE             KERNEL-VERSI
ON              CONTAINER-RUNTIME
master    Ready    control-plane   20d   v1.36.4+k3s1   192.168.159.10   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-ge
neric (amd64)   containerd://2.3.4-k3s1.36
worker1   Ready    <none>          13d   v1.36.5+k3s1   192.168.159.12   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-ge
neric (amd64)   containerd://2.3.4-k3s1.36
worker2   Ready    <none>          13d   v1.36.5+k3s1   192.168.159.11   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-ge
neric (amd64)   containerd://2.3.4-k3s1.36
```

*Vérifier cluster :

```
arij@master:~$ kubectl get pods -A
NAMESPACE     NAME                                      READY   STATUS      RESTARTS        AGE
kube-system   coredns-54996dc9b4-dctrx                  1/1     Running     8 (10m ago)     20d
kube-system   helm-install-traefik-crd-ds7vk            0/1     Completed   0               20d
kube-system   helm-install-traefik-l2h52                0/1     Completed   2 (20d ago)     20d
kube-system   local-path-provisioner-77b9867795-h86xg   1/1     Running     5 (10m ago)     20d
kube-system   metrics-server-6dc596dfb8-tmrw8           1/1     Running     13 (10m ago)    20d
kube-system   svclb-traefik-1c697864-47r9z              2/2     Running     10 (10m ago)    20d
kube-system   svclb-traefik-1c697864-cxtxh              2/2     Running     4 (9m53s ago)   13d
kube-system   svclb-traefik-1c697864-n5bsr              2/2     Running     4 (9m50s ago)   13d
kube-system   traefik-59b7647586-szfcq                  1/1     Running     7 (10m ago)     20d
lab2-app      web-deployment-745f7cf7b-2qtp6            1/1     Running     1 (10m ago)     6d18h
lab2-app      web-deployment-745f7cf7b-48qnm            1/1     Running     1 (10m ago)     6d18h
lab2-app      web-deployment-745f7cf7b-fn9g6            1/1     Running     2 (10m ago)     13d
lab2-app      web-deployment-745f7cf7b-w7rd7            1/1     Running     1 (10m ago)     6d18h
lab2-app      web-deployment-745f7cf7b-xtf8v            1/1     Running     1 (10m ago)     6d18h
mp1-arij      api-7b989b589-8nscd                       1/1     Running     1 (10m ago)     6d16h
mp1-arij      api-7b989b589-lflcr                       1/1     Running     1 (9m50s ago)   6d16h
mp1-arij      api-7b989b589-qg27p                       1/1     Running     1 (9m53s ago)   6d16h
mp1-arij      frontend-7f4664b7cd-kv82z                 1/1     Running     1 (9m50s ago)   6d17h
mp1-arij      frontend-7f4664b7cd-l8jcq                 1/1     Running     1 (9m53s ago)   6d17h
tp-k8s        broken                                    1/1     Running     1 (9m53s ago)   6d1h
tp-k8s        demo-7b48f66b47-cdp56                     1/1     Running     1 (9m50s ago)   6d1h
tp-k8s        web-65c9c67844-t8vh8                      1/1     Running     1 (9m53s ago)   6d1h
```

*Vérifier curl.exe :

```
C:\Users\arijb>curl.exe --version
curl 8.21.0 (Windows) libcurl/8.21.0 Schannel zlib/1.3.2 WinIDN WinLDAP
Release-Date: 2026-06-24
Protocols: dict file ftp ftps gopher gophers http https imap imaps ipfs ipns ldap ldaps mqtt mqtts pop3 pop3s smtp smtps
 telnet tftp ws wss
Features: alt-svc AsynchDNS HSTS HTTPS-proxy IDN IPv6 Kerberos Largefile libz SPNEGO SSL SSPI threadsafe Unicode UnixSoc
kets

C:\Users\arijb>
```

Étape 1 – Préparer le namespace et déployer l'application

1.1 Créer le namespace

```
arij@master:~/lab3.1$ kubectl create namespace tp-k8s
Error from server (AlreadyExists): namespaces "tp-k8s" already exists
```

1.2 Basculer le contexte courant sur ce namespace

```
arij@master:~/lab3.1$ kubectl config set-context --current --namespace=tp-k8s
Context "default" modified.
```

```
arij@master:~/lab3.1$ kubectl config view --minify | grep namespace
    namespace: tp-k8s
arij@master:~/lab3.1$
```

1.3 Écrire le manifeste du Deployment

```
arij@master:~/lab3.1$ cat <<'EOF' > podinfo.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo
  labels:
    app: podinfo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: podinfo
  template:
    metadata:
      labels:
        app: podinfo
    spec:
      containers:
        - name: podinfo
          image: ghcr.io/stefanprodan/podinfo:6.5.4
          ports:
            - containerPort: 9898
EOF
```

1.4 Appliquer le manifeste

```
arij@master:~/lab3.1$ kubectl apply -f podinfo.yaml
deployment.apps/podinfo created
```

1.5 Vérifier les Pods

```
arij@master:~/lab3.1$ kubectl get pods -o wide
NAME                      READY   STATUS    RESTARTS      AGE    IP           NODE      NOMINATED NODE   READINESS GATES
broken                    1/1     Running   1 (23m ago)   6d1h   10.42.3.35   worker2   <none>           <none>
demo-7b48f66b47-cdp56     1/1     Running   1 (23m ago)   6d1h   10.42.1.25   worker1   <none>           <none>
podinfo-f6698f896-j5dz8   1/1     Running   0             22s    10.42.3.36   worker2   <none>           <none>
podinfo-f6698f896-x7q4d   1/1     Running   0             22s    10.42.1.27   worker1   <none>           <none>
podinfo-f6698f896-zvgt2   1/1     Running   0             22s    10.42.0.54   master    <none>           <none>
web-65c9c67844-t8vh8      1/1     Running   1 (23m ago)   6d1h   10.42.3.34   worker2   <none>           <none>
```

Étape 2 _Créer un Service ClusterIP :

2.1 Créer le fichier YAML du Service

```
arij@master:~/lab3.1$ cat <<'EOF' > podinfo-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: podinfo-svc
spec:
  type: ClusterIP
  selector:
    app: podinfo
  ports:
    - port: 80
      targetPort: 9898
EOF
arij@master:~/lab3.1$
```

2.2 Appliquer le Service

```
arij@master:~/lab3.1$ kubectl apply -f podinfo-svc.yaml
service/podinfo-svc created
```

2.3 Vérifier le Service

```
arij@master:~/lab3.1$ kubectl get service podinfo-svc
NAME          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
podinfo-svc   ClusterIP   10.43.186.72   <none>        80/TCP    41s
```

2.4 Vérifier les Endpoints

```
arij@master:~/lab3.1$ kubectl get endpoints podinfo-svc
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS                                            AGE
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898   68s
arij@master:~/lab3.1$
```

2.5 Comparer avec les Pods

```
arij@master:~/lab3.1$ kubectl get pods -o wide
NAME                      READY   STATUS    RESTARTS      AGE    IP           NODE      NOMINATED NODE   READINESS GATES
broken                    1/1     Running   1 (28m ago)   6d1h   10.42.3.35   worker2   <none>           <none>
demo-7b48f66b47-cdp56     1/1     Running   1 (28m ago)   6d1h   10.42.1.25   worker1   <none>           <none>
podinfo-f6698f896-j5dz8   1/1     Running   0             6m1s   10.42.3.36   worker2   <none>           <none>
podinfo-f6698f896-x7q4d   1/1     Running   0             6m1s   10.42.1.27   worker1   <none>           <none>
podinfo-f6698f896-zvgt2   1/1     Running   0             6m1s   10.42.0.54   master    <none>           <none>
web-65c9c67844-t8vh8      1/1     Running   1 (28m ago)   6d1h   10.42.3.34   worker2   <none>           <none>
arij@master:~/lab3.1$
```

Étape_3 :Joindre le Service par son nom DNS

3.1 Test avec le nom court

```
arij@master:~/lab3.1$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://podinfo-svc
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
Session ended, resume using 'kubectl attach test -c test -n tp-k8s -i -t' command
pod "test" deleted from tp-k8s namespace
```

3.2 Test avec le nom complet (FQDN)

```
arij@master:~/lab3.1$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://podinfo-svc.tp-k8s.svc.clust
er.local
{
  "hostname": "podinfo-f6698f896-j5dz8",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
 through the command prompt.
If you don't see a command prompt, try pressing enter.
Session ended, resume using 'kubectl attach test -c test -n tp-k8s -i -t' command
pod "test" deleted from tp-k8s namespace
```

3.3 Test de résolution DNS avec nslookup

```
arij@master:~/lab3.1$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- nslookup podinfo-svc
Server:         10.43.0.10
Address:        10.43.0.10:53

Name:   podinfo-svc.tp-k8s.svc.cluster.local
Address: 10.43.186.72

** server can't find podinfo-svc.svc.cluster.local: NXDOMAIN

** server can't find podinfo-svc.cluster.local: NXDOMAIN

** server can't find podinfo-svc.svc.cluster.local: NXDOMAIN

** server can't find podinfo-svc.cluster.local: NXDOMAIN


All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
warning: couldn't attach to pod/test, falling back to streaming logs: unable to upgrade connection: container test not found in pod t
est_tp-k8s
Server:         10.43.0.10
Address:        10.43.0.10:53

Name:   podinfo-svc.tp-k8s.svc.cluster.local
Address: 10.43.186.72

** server can't find podinfo-svc.svc.cluster.local: NXDOMAIN

** server can't find podinfo-svc.cluster.local: NXDOMAIN

** server can't find podinfo-svc.svc.cluster.local: NXDOMAIN
```

Étape_4 : Observer la répartition de charge

```
arij@master:~/lab3.1$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- sh -c 'for i in 1 2 3 4 5 6; do wget -qO- htt
p://podinfo-svc | grep hostname; done; sleep 2'
  "hostname": "podinfo-f6698f896-zvgt2",
  "hostname": "podinfo-f6698f896-x7q4d",
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
  "hostname": "podinfo-f6698f896-zvgt2",
Session ended, resume using 'kubectl attach test -c test -n tp-k8s -i -t' command
pod "test" deleted from tp-k8s namespace
arij@master:~/lab3.1$
```

Étape 5 : Suivre les Pods avec les endpoints

1.Terminal 1 :lancer surveillance :

```
arij@master:~/lab3.1$ kubectl get endpoints podinfo-svc -w
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS                                          AGE
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898   11m
```

2.Terminal 2 : Passer à 5 réplicas :

```
arij@master:~/lab3.1$ kubectl scale deployment podinfo --replicas=5
deployment.apps/podinfo scaled
```

3.Terminal2 : Vérifier les endpoints

```
arij@master:~/lab3.1$ kubectl get endpoints podinfo-svc
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS                                                     AGE
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.1.32:9898 + 2 more...   12m
```

4.Terminal2 : Supprimer un pod au hasard :

```
arij@master:~/lab3.1$ POD=$(kubectl get pods -l app=podinfo -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $POD
pod "podinfo-f6698f896-6w9pb" deleted from tp-k8s namespace
```

5.Terminal2 : Revenir à 3 réplicas :

```
arij@master:~/lab3.1$ kubectl scale deployment podinfo --replicas=3
deployment.apps/podinfo scaled
```

6.Observation sur Terminal 1

```
arij@master:~/lab3.1$ kubectl get endpoints podinfo-svc -w
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS                                            AGE
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898      11m
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.1.32:9898 + 1 more...   11m
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.1.32:9898 + 2 more...   11m
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898 + 1 more...   15m
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.1.33:9898 + 2 more...   15m
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.1.33:9898 + 1 more...   15m
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898               15m
```

Étape_6 : Diagnostiquer un sélecteur incorrect :

6.1 Casser le sélecteur :

```
^Carij@master:~/lab3.1kubectl set selector service podinfo-svc app=autrere
service/podinfo-svc selector updated
```

6.2 Constater les dégâts :

```
arij@master:~/lab3.1$ kubectl get endpoints podinfo-svc
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS   AGE
podinfo-svc   <none>      17m
```

6.3 Tester l'appel

```
arij@master:~/lab3.1$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- wget -T 3 -qO- http://podinfo-svc
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
wget: can't connect to remote host (10.43.186.72): Connection refused
Session ended, resume using 'kubectl attach test -c test -n tp-k8s -i -t' command
pod "test" deleted from tp-k8s namespace
pod tp-k8s/test terminated (Error)
```

6.4 Vérifier le sélecteur (pour diagnostiquer) :

```
arij@master:~/lab3.1$ kubectl describe service podinfo-svc | grep Selector
Selector:                 app=autre
arij@master:~/lab3.1$
```

6.5 Réparer le sélecteur :

```
arij@master:~/lab3.1$ kubectl set selector service podinfo-svc app=podinfo
service/podinfo-svc selector updated
```

6.6 Vérifier le retour à la normale :

```
arij@master:~/lab3.1$ kubectl get endpoints podinfo-svc
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS                                            AGE
podinfo-svc   10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898      19m
```

Étape _7 : Exposer hors du cluster avec NodePort

1.Créer le fichier YAML

```
arij@master:~/lab3.1$ cat <<'EOF' > podinfo-nodeport.yaml
apiVersion: v1
kind: Service
metadata:
  name: podinfo-nodeport
spec:
  type: NodePort
  selector:
    app: podinfo
  ports:
    - port: 80
      targetPort: 9898
EOF
```

2.Appliquer le Service :

```
arij@master:~/lab3.1$ kubectl apply -f podinfo-nodeport.yaml
service/podinfo-nodeport created
arij@master:~/lab3.1$
```

3.Récupérer le NodePort attribué

```
arij@master:~/lab3.1$ kubectl get service podinfo-nodeport
NAME              TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
podinfo-nodeport   NodePort   10.43.150.59   <none>        80:32284/TCP   26s
```

4.Vérifier nodes :

```
arij@master:~/lab3.1$ kubectl get nodes -o wide
NAME      STATUS   ROLES           AGE   VERSION        INTERNAL-IP      EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION
   CONTAINER-RUNTIME
master    Ready    control-plane   20d   v1.36.4+k3s1   192.168.159.10   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-generic (amd64)
   containerd://2.3.4-k3s1.36
worker1   Ready    <none>          13d   v1.36.5+k3s1   192.168.159.12   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-generic (amd64)
   containerd://2.3.4-k3s1.36
worker2   Ready    <none>          13d   v1.36.5+k3s1   192.168.159.11   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-generic (amd64)
   containerd://2.3.4-k3s1.36
```

5.Exposer hors du cluster avec NodePort

```
C:\Users\arijb>curl.exe http://192.168.159.10:32284
{
  "hostname": "podinfo-f6698f896-j5dz8",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}
C:\Users\arijb>curl.exe http://192.168.159.11:32284
{
  "hostname": "podinfo-f6698f896-x7q4d",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}
C:\Users\arijb>curl.exe http://192.168.159.12:32284
{
  "hostname": "podinfo-f6698f896-zvgt2",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
```

Étape_8 : Exposer avec LoadBalancer (ServiceLB)

1.Créer le 3eme service :

```
arij@master:~/lab3.1$ cat <<'EOF' > podinfo-lb.yaml
apiVersion: v1
kind: Service
metadata:
  name: podinfo-lb
spec:
  type: LoadBalancer
  selector:
    app: podinfo
  ports:
    - port: 8080
      targetPort: 9898
EOF
```

2. Appliquer

```
arij@master:~/lab3.1$ kubectl apply -f podinfo-lb.yaml
service/podinfo-lb created
```

3. Vérifier l'EXTERNAL-IP et les Pods svclb

```
arij@master:~/lab3.1$ kubectl get service podinfo-lb
NAME         TYPE           CLUSTER-IP     EXTERNAL-IP                                        PORT(S)          AGE
podinfo-lb   LoadBalancer   10.43.67.255   192.168.159.10,192.168.159.11,192.168.159.12   8080:31928/TCP   50s
arij@master:~/lab3.1$
```

4.Vérifier pods

```
arij@master:~/lab3.1$ kubectl get pods -n kube-system | grep svclb
svclb-podinfo-lb-a3910a28-kg4vq           1/1     Running     0              106s
svclb-podinfo-lb-a3910a28-mf545           1/1     Running     0              106s
svclb-podinfo-lb-a3910a28-tbjwk           1/1     Running     0              107s
svclb-traefik-1c697864-47r9z              2/2     Running     10 (56m ago)   20d
svclb-traefik-1c697864-cxtxh              2/2     Running     4 (56m ago)    13d
svclb-traefik-1c697864-n5bsr              2/2     Running     4 (56m ago)    13d
arij@master:~/lab3.1$
```

5.Test final:

```
C:\Users\arijb>curl.exe http://192.168.159.10:8080
{
  "hostname": "podinfo-f6698f896-j5dz8",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}
C:\Users\arijb>curl.exe http://192.168.159.11:8080
{
  "hostname": "podinfo-f6698f896-j5dz8",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}
C:\Users\arijb>curl.exe http://192.168.159.12:8080
{
  "hostname": "podinfo-f6698f896-x7q4d",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
```

Étape 9 : Joindre un Service depuis un autre namespace

1. Créer un nouveau namespace

```
arij@master:~/lab3.1$ kubectl create namespace tp-autre
namespace/tp-autre created
arij@master:~/lab3.1$
```

2.Tester avec le nom court

```
arij@master:~/lab3.1$ kubectl run test -n tp-autre --rm -it --image=busybox:1.36 --restart=Never -- wget -T 3 -qO- http://podinfo-svc
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
Session ended, resume using 'kubectl attach test -c test -n tp-autre -i -t' command
pod "test" deleted from tp-autre namespace
pod tp-autre/test terminated (Error)
```

3. Tester avec le nom qualifié

```
arij@master:~/lab3.1$ kubectl run test -n tp-autre --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://podinfo-svc.tp-k
8s
{
  "hostname": "podinfo-f6698f896-x7q4d",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
 through the command prompt.
If you don't see a command prompt, try pressing enter.
warning: couldn't attach to pod/test, falling back to streaming logs: Internal error occurred: Internal error occurred: error attachi
ng to container: container is in CONTAINER_EXITED state
{
  "hostname": "podinfo-f6698f896-x7q4d",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}pod "test" deleted from tp-autre namespace
```

Lab 3.2 : Ingress avec Traefik

Étape_1 : Déployer les deux applications :

1.Créer le fichier apps.yaml

```
arij@master:~/lab3.2$ cat <<'EOF' > apps.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.26-alpine
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: apps/v1
```

2 . Appliquer

```
arij@master:~/lab3.2$ kubectl apply -f apps.yaml
deployment.apps/web configured
service/web-svc unchanged
deployment.apps/api created
service/api-svc created
```

3.vérifier

```
arij@master:~/lab3.2$ kubectl get deployments,services
NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api          3/3     3            3           6s
deployment.apps/demo         1/1     1            1           6d2h
deployment.apps/podinfo      3/3     3            3           60m
deployment.apps/web          2/2     2            2           6d2h

NAME                        TYPE           CLUSTER-IP     EXTERNAL-IP                                      PORT(S)          AGE
service/api-svc             ClusterIP      10.43.20.42    <none>                                           80/TCP           6s
service/podinfo-lb          LoadBalancer   10.43.67.255   192.168.159.10,192.168.159.11,192.168.159.12     8080:31928/TCP   28m
service/podinfo-nodeport    NodePort       10.43.150.59   <none>                                           80:32284/TCP     34m
service/podinfo-svc         ClusterIP      10.43.186.72   <none>                                           80/TCP           56m
service/web-svc             ClusterIP      10.43.71.54    <none>                                           80/TCP           6d2h
arij@master:~/lab3.2$ kubectl get endpoints
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME              ENDPOINTS                                            AGE
api-svc           10.42.0.56:9898,10.42.1.40:9898,10.42.3.39:9898      14s
podinfo-lb        10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898      28m
podinfo-nodeport  10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898      34m
podinfo-svc       10.42.0.54:9898,10.42.1.27:9898,10.42.3.36:9898      56m
web-svc           10.42.1.38:80,10.42.3.40:80                          6d2h
```

Étape_2 : Repérer Traefik :

2.1 Vérifier le pod Traefik

```
arij@master:~/lab3.2$ kubectl get pods -n kube-system | grep traefik
helm-install-traefik-crd-ds7vk            0/1     Completed   0              20d
helm-install-traefik-l2h52                0/1     Completed   2 (20d ago)    20d
svclb-traefik-1c697864-47r9z              2/2     Running     10 (85m ago)   20d
svclb-traefik-1c697864-cxtxh              2/2     Running     4 (85m ago)    13d
svclb-traefik-1c697864-n5bsr              2/2     Running     4 (85m ago)    13d
traefik-59b7647586-szfcq                  1/1     Running     7 (85m ago)    20d
```

2.2 Vérifier le Service Traefik

```
arij@master:~/lab3.2$ kubectl get service traefik -n kube-system
NAME      TYPE           CLUSTER-IP      EXTERNAL-IP                                  PORT(S)                      AGE
traefik   LoadBalancer   10.43.246.241   192.168.159.10,192.168.159.11,192.168.159.12   80:30262/TCP,443:31663/TCP   20d
```

2.3 Vérifier l'IngressClass

```
arij@master:~/lab3.2$ kubectl get ingressclass
NAME                CONTROLLER                      PARAMETERS   AGE
traefik (default)   traefik.io/ingress-controller   <none>       20d
```

3.3.Vérifier nodes :

```
arij@master:~/lab3.2$ kubectl get nodes -o wide
NAME      STATUS   ROLES           AGE   VERSION        INTERNAL-IP      EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION
   CONTAINER-RUNTIME
master    Ready    control-plane   20d   v1.36.4+k3s1   192.168.159.10   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-generic (amd64)
   containerd://2.3.4-k3s1.36
worker1   Ready    <none>          13d   v1.36.5+k3s1   192.168.159.12   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-generic (amd64)
   containerd://2.3.4-k3s1.36
worker2   Ready    <none>          13d   v1.36.5+k3s1   192.168.159.11   <none>        Ubuntu 24.04.3 LTS   6.8.0-146-generic (amd64)
   containerd://2.3.4-k3s1.36
```

Étape_3 : Créer un Ingress par nom d'hôte

3.1 Créer le fichier YAML

```
arij@master:~/lab3.2$ cat <<'EOF' > ingress-hotes.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: apps-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: web.k3s.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-svc
                port:
                  number: 80
    - host: api.k3s.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api-svc
                port:
                  number: 80
EOF
```

3.2 Appliquer l'Ingress

```
arij@master:~/lab3.2$ kubectl apply -f ingress-hotes.yaml
ingress.networking.k8s.io/apps-ingress created
```

3.3 Vérifier l'Ingress

```
arij@master:~/lab3.2$ kubectl get ingress
NAME           CLASS     HOSTS                       ADDRESS                                      PORTS   AGE
apps-ingress   traefik   web.k3s.local,api.k3s.local   192.168.159.10,192.168.159.11,192.168.159.12   80      17s
```

3.4 Inspecter les règles

```
arij@master:~/lab3.2$ kubectl describe ingress apps-ingress
Name:             apps-ingress
Labels:           <none>
Namespace:        tp-k8s
Address:          192.168.159.10,192.168.159.11,192.168.159.12
Ingress Class:    traefik
Default backend:  <default>
Rules:
  Host           Path  Backends
  ----           ----  --------
  web.k3s.local
                 /   web-svc:80 (10.42.1.38:80,10.42.3.40:80)
  api.k3s.local
                 /   api-svc:80 (10.42.3.39:9898,10.42.0.56:9898,10.42.1.40:9898)
Annotations:     <none>
Events:          <none>
```

Étape_4 : Tester avec l'en-tête Host

4.1 Test de l'application web (Nginx

```
C:\Users\arijb>curl.exe -H "Host: web.k3s.local" http://192.168.159.10
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

4.2 Test de l'application API (Podinfo)

```
C:\Users\arijb>curl.exe -H "Host: api.k3s.local" http://192.168.159.10
{
  "hostname": "api-7779574f48-2ddtl",
  "version": "6.5.4",
  "revision": "33dac1ba40f73555725fbf620bf3b4f6f1a5ad89",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.5.4",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.5",
  "num_goroutine": "6",
  "num_cpu": "2"
}
```

4.3 Test sur un autre nœud

```
C:\Users\arijb>curl.exe -H "Host: web.k3s.local" http://192.168.159.12
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

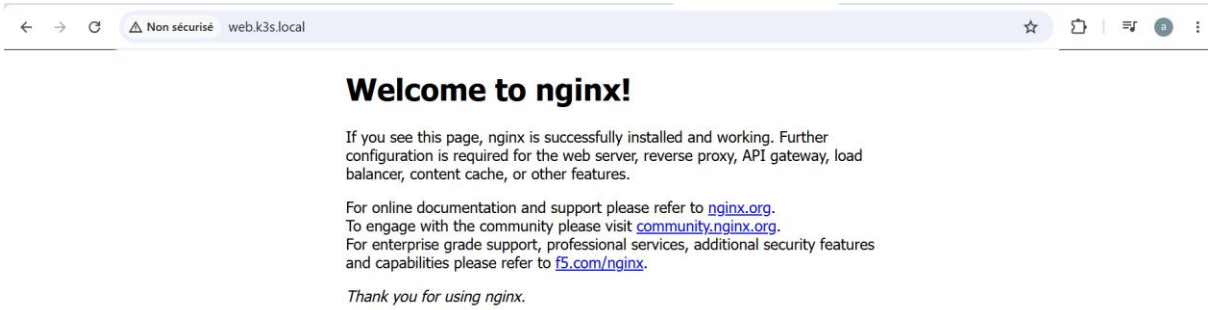
C:\Users\arijb>curl.exe -H "Host: api.k3s.local" http://192.168.159.12
{
  "hostname": "api-7779574f48-vtblg",
  "version": "6.5.4",
```

Étape_5 : Utiliser de vrais noms d'hôte

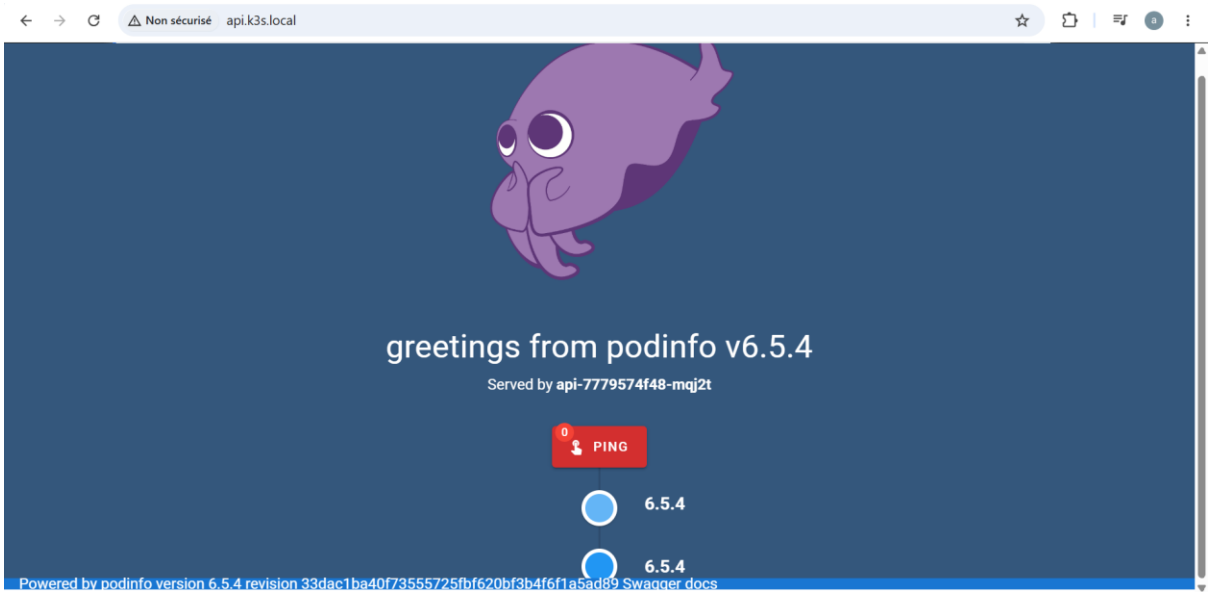
1.Modifier fichier hosts :

```
# End of section
192.168.159.10 web.k3s.local api.k3s.local
192.168.159.21 portfolio.coolify.local
192.168.159.10  web.k3s.local api.k3s.local site.k3s.local
```

4. Tester :



Et :



Étape_6 : Router par chemin :

1. crée le fichier ingress-chemins.yaml

```
arij@master:~/lab3.2$ cat <<'EOF' > ingress-chemins.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: site-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: site.k3s.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-svc
                port:
                  number: 80
          - path: /version
            pathType: Prefix
            backend:
              service:
                name: api-svc
                port:
                  number: 80
EOF
```

2.Appliquer :

```
arij@master:~/lab3.2$ kubectl apply -f ingress-chemins.yaml
ingress.networking.k8s.io/site-ingress created
arij@master:~/lab3.2$ kubectl get ingress
NAME           CLASS     HOSTS                    ADDRESS                                          PORTS   AGE
apps-ingress   traefik   web.k3s.local,api.k3s.local   192.168.159.10,192.168.159.11,192.168.159.12   80      9m54s
site-ingress   traefik   site.k3s.local           192.168.159.10,192.168.159.11,192.168.159.12   80      6s
```

3.Tester depuis Windows



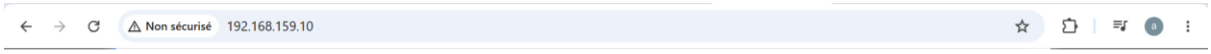
et



Étape_7 : Distinguer les erreurs :

1.Hôte inconnu de l'Ingress (Erreur de routage)

```
arij@master:~/lab3.2$ curl.exe -i -H "Host: inconnu.k3s.local" http://192.168.159.10
curl.exe: command not found
arij@master:~/lab3.2$
```



2.Chemin inexistant dans Nginx (Erreur d'application)

```
C:\Users\arijb>curl.exe -i http://web.k3s.local/nexiste-pas
HTTP/1.1 404 Not Found
Content-Length: 153
Content-Type: text/html
Date: Sun, 18 Oct 2026 13:22:59 GMT
Server: nginx/1.31.6

<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.31.6</center>
</body>
</html>
```

3.Service sans aucun Pod (Erreur de backend)

*sur ubuntu

```
arij@master:~/lab3.2$ kubectl scale deployment api --replicas=0
deployment.apps/api scaled
```

Sur cmd Windows :

```
C:\Users\arijb>curl.exe -i http://api.k3s.local
HTTP/1.1 503 Service Unavailable
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Date: Sun, 18 Oct 2026 13:19:24 GMT
Content-Length: 20

no available server
```

4.Réparer l'API

```
arij@master:~/lab3.2$ kubectl scale deployment api --replicas=3
deployment.apps/api scaled
arij@master:~/lab3.2$
```

5.Vérifier sur Windows :

```
C:\Users\arijb>curl.exe http://api.k3s.local
{
  "hostname": "api-7b989b589-8nscd",
  "version": "6.6.0",
  "revision": "357009a86331a987811fefc11be1350058da33fc",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.6.0",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.21.7",
  "num_goroutine": "6",
  "num_cpu": "2"
}
```

Lab 4.1 : Externaliser la configuration

Étape 1 — Préparer le namespace :

```
arij@master:~$ cd lab4.1
arij@master:~/lab4.1$ kubectl create namespace tp-k8s
namespace/tp-k8s created
arij@master:~/lab4.1$ kubectl config set-context --current --namespace=tp-k8s
Context "default" modified.
arij@master:~/lab4.1$
```

Étape 2 — Créer un ConfigMap :

```
arij@master:~/lab4.1$ cat <<'EOF' > app-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: dev
  APP_COLOR: bleu
  LOG_LEVEL: info
EOF
```

```
arij@master:~/lab4.1$ kubectl apply -f app-config.yaml
configmap/app-config created
arij@master:~/lab4.1$ kubectl get configmap app-config
NAME         DATA   AGE
app-config   3      9s
arij@master:~/lab4.1$ kubectl describe configmap app-config
Name:         app-config
Namespace:    tp-k8s
Labels:       <none>
Annotations:  <none>

Data
====
APP_COLOR:
----
bleu

APP_ENV:
----
dev

LOG_LEVEL:
----
info


BinaryData
====

Events:  <none>
arij@master:~/lab4.1$
```

Étape 3 — Créer un Secret

```
arij@master:~/lab4.1$ kubectl create secret generic db-credentials \
  --from-literal=DB_USER=appuser \
  --from-literal=DB_PASSWORD='ChangeMe-123'
secret/db-credentials created
arij@master:~/lab4.1$ kubectl get secret db-credentials
NAME             TYPE     DATA   AGE
db-credentials   Opaque   2      15s
arij@master:~/lab4.1$ kubectl describe secret db-credentials
Name:         db-credentials
Namespace:    tp-k8s
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
DB_PASSWORD:  12 bytes
DB_USER:      7 bytes
arij@master:~/lab4.1$ kubectl get secret db-credentials -o yaml
apiVersion: v1
data:
  DB_PASSWORD: Q2hhbmdlTWUtMTIz
  DB_USER: YXBwdXNlcg==
kind: Secret
metadata:
  creationTimestamp: "2026-10-12T10:00:52Z"
  name: db-credentials
  namespace: tp-k8s
  resourceVersion: "28626"
  uid: c5a0efef-9d17-4450-b2e5-bcade01720f6
type: Opaque
arij@master:~/lab4.1$
```

a.Décoder le mot de passe (Point de contrôle) :

```
arij@master:~/lab4.1$ kubectl get secret db-credentials -o jsonpath='{.data.DB_PASSWORD}' | base64 -d; echo
ChangeMe-123
arij@master:~/lab4.1$
```

b.Générer le fichier manifeste sans créer la ressource (Dry-run) :

```
arij@master:~/lab4.1$ kubectl create secret generic db-credentials \
  --from-literal=DB_USER=appuser \
  --from-literal=DB_PASSWORD='ChangeMe-123' \
  --dry-run=client -o yaml > secret.yaml

cat secret.yaml
apiVersion: v1
data:
  DB_PASSWORD: Q2hhbmdlTWUtMTIz
  DB_USER: YXBwdXNlcg==
kind: Secret
metadata:
  name: db-credentials
arij@master:~/lab4.1$
```

Étape 4 — Injecter la configuration dans un Deployment

```
arij@master:~/lab4.1$ cat <<'EOF' > demo.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
    spec:
      containers:
        - name: demo
          image: nginx:1.26-alpine
          env:
            - name: NIVEAU_LOG                # une clé, sous un autre nom
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: LOG_LEVEL
            - name: DB_USER                   # clés du Secret
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: DB_USER
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials
```

```
arij@master:~/lab4.1$ kubectl apply -f demo.yaml
deployment.apps/demo created
arij@master:~/lab4.1$ kubectl rollout status deployment/demo
deployment "demo" successfully rolled out
arij@master:~/lab4.1$ kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
demo-5cdc7fd475-vdxwz   1/1     Running   0          18s
arij@master:~/lab4.1$
```

Étape 5 — Vérifier l'injection

```
arij@master:~/lab4.1$ kubectl exec deploy/demo -- env | grep -E 'APP_|NIVEAU_|DB_'
kubectl exec deploy/demo -- ls /etc/app /etc/secret
kubectl exec deploy/demo -- cat /etc/app/APP_COLOR
kubectl exec deploy/demo -- cat /etc/secret/DB_PASSWORD; echo
DB_PASSWORD=ChangeMe-123
APP_COLOR=bleu
APP_ENV=dev
NIVEAU_LOG=info
DB_USER=appuser
/etc/app:
APP_COLOR
APP_ENV
LOG_LEVEL

/etc/secret:
DB_PASSWORD
DB_USER
bleuChangeMe-123
arij@master:~/lab4.1$
```

Étape 6 — Modifier la configuration et observer

```
arij@master:~/lab4.1$ sed -i 's|APP_COLOR: bleu|APP_COLOR: vert|' app-config.yaml
kubectl apply -f app-config.yaml

kubectl exec deploy/demo -- sh -c 'echo "variable : $APP_COLOR"; echo "fichier   : $(cat /etc/app/APP_COLOR)"'
configmap/app-config configured
variable : bleu
fichier   : bleu
arij@master:~/lab4.1$ kubectl exec deploy/demo -- sh -c 'echo "variable : $APP_COLOR"; echo "fichier   : $(cat /etc/app/APP_COLOR)"'
variable : bleu
fichier   : vert
arij@master:~/lab4.1$ kubectl rollout restart deployment/demo
kubectl rollout status deployment/demo
kubectl exec deploy/demo -- sh -c 'echo "variable : $APP_COLOR"; echo "fichier   : $(cat /etc/app/APP_COLOR)"'
deployment.apps/demo restarted
Waiting for deployment "demo" rollout to finish: 0 out of 1 new replicas have been updated...
Waiting for deployment "demo" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "demo" rollout to finish: 1 old replicas are pending termination...
deployment "demo" successfully rolled out
variable : vert
fichier   : vert
arij@master:~/lab4.1$
```

Étape 7 — Monter un fichier de configuration dans nginx

```
^Carij@master:~/lab4.1$ cat <<'EOF' > web.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf
data:
  default.conf: |
    server {
      listen 80;
      default_type text/plain;
      location / {
        return 200 'Bonjour depuis le ConfigMap v1\n';
      }
    }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.26-alpine
```

```
arij@master:~/lab4.1$ kubectl apply -f web.yaml
kubectl rollout restart deployment/web
kubectl rollout status deployment/web
configmap/nginx-conf unchanged
deployment.apps/web unchanged
service/web-svc unchanged
deployment.apps/web restarted
Waiting for deployment "web" rollout to finish: 0 out of 1 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
deployment "web" successfully rolled out
arij@master:~/lab4.1$
```

Test :

```
arij@master:~/lab4.1$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://web-svc
Bonjour depuis le ConfigMap v1
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
Session ended, resume using 'kubectl attach test -c test -n tp-k8s -i -t' command
pod "test" deleted from tp-k8s namespace
arij@master:~/lab4.1$
```

1. Modifiez le ConfigMap pour passer en v2 :

```
arij@master:~/lab4.1$ sed -i 's|ConfigMap v1|ConfigMap v2|' web.yaml
kubectl apply -f web.yaml
configmap/nginx-conf configured
deployment.apps/web unchanged
service/web-svc unchanged
arij@master:~/lab4.1$ kubectl exec deploy/web -- cat /etc/nginx/conf.d/default.conf
sleep 90
server {
  listen 80;
  default_type text/plain;
  location / {
    return 200 'Bonjour depuis le ConfigMap v1\n';
  }
}
arij@master:~/lab4.1$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://web-svc
Bonjour depuis le ConfigMap v1
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
warning: couldn't attach to pod/test, falling back to streaming logs: Internal error occurred: Internal error occurred: error attachi
ng to container: failed to load task: no running task found: task 7213a34ff9baab7548ed3e893b5917b1ca9d74d016fd0104b72cf19653dc1be5 no
t found
Bonjour depuis le ConfigMap v1
pod "test" deleted from tp-k8s namespace
```

4. Redémarrez le deployment pour forcer la mise à jour :

```
arij@master:~/lab4.1$ kubectl rollout restart deployment/web
kubectl rollout status deployment/web
kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://web-svc
deployment.apps/web restarted
Waiting for deployment "web" rollout to finish: 0 out of 1 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
deployment "web" successfully rolled out
Bonjour depuis le ConfigMap v2
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
Warning: couldn't attach to pod/test, falling back to streaming logs: unable to upgrade connection: container test not found in pod t
est_tp-k8s
Bonjour depuis le ConfigMap v2
pod "test" deleted from tp-k8s namespace
arij@master:~/lab4.1$
```

Étape 8 — Diagnostiquer une configuration manquante

1. Créez le fichier broken.yaml avec nano (pour éviter les coupures) :

```
 GNU nano 7.2                                                broken.yaml *
apiVersion: v1
kind: Pod
metadata:
  name: broken
spec:
  containers:
    - name: app
      image: nginx:1.26-alpine
      envFrom:
        - configMapRef:
            name: config-manquant
```

2.Appliquez le fichier et observez l'erreur :

```
arij@master:~/lab4.1$ kubectl apply -f broken.yaml
kubectl get pod broken
kubectl describe pod broken | tail -n 8
pod/broken created
NAME     READY   STATUS              RESTARTS   AGE
broken   0/1     ContainerCreating   0          1s
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  0s    default-scheduler  Successfully assigned tp-k8s/broken to worker2
arij@master:~/lab4.1$
```

3.Corrigez le problème en créant le ConfigMap manquant :

```
arij@master:~/lab4.1$ kubectl create configmap config-manquant --from-literal=EXEMPLE=1
configmap/config-manquant created
arij@master:~/lab4.1$
```

4.Surveillez le Pod pour voir s'il démarre tout seul :

```
arij@master:~/lab4.1$ kubectl get pod broken -w
NAME     READY   STATUS    RESTARTS   AGE
broken   1/1     Running   0          2m32s
```

Lab 4.2 : Maîtriser les ressources

Étape_1 : Lire la capacité du cluster :

1.Voir les capacités (CPU / Mémoire) :

```
arij@master:~/lab4.2$ kubectl get nodes -o custom-columns=NOM:.metadata.name,CPU:.status.allocatable.cpu,MEMOIRE:.status.allocatable.
memory
NOM       CPU   MEMOIRE
master    2     3960856Ki
worker1   2     3960852Ki
worker2   2     3960852Ki
arij@master:~/lab4.2$
```

2. Voir la consommation en temps réel

```
arij@master:~/lab4.2$ kubectl top nodes
NAME      CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
master    126m         6%       1659Mi          42%
worker1   45m          2%       740Mi           19%
worker2   43m          2%       725Mi           18%
arij@master:~/lab4.2$
```

3.Voir les ressources déjà réservées sur un nœud

```
arij@master:~/lab4.2$ kubectl describe node worker1 | grep -A8 "Allocated resources"
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests  Limits
  --------           --------  ------
  cpu                0 (0%)    0 (0%)
  memory             0 (0%)    0 (0%)
  ephemeral-storage  0 (0%)    0 (0%)
  hugepages-1Gi      0 (0%)    0 (0%)
  hugepages-2Mi      0 (0%)    0 (0%)
```

Étape_2 : Déclarer des requests et des limits

1.Créer le fichier app.yaml

```
arij@master:~/lab4.2$ cat <<'EOF' > app.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          image: nginx:1.26-alpine
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 128Mi
EOF
```

2. Appliquer & Vérifier

```
arij@master:~/lab4.2$ kubectl apply -f app.yaml
kubectl rollout status deployment/app
kubectl get pods -o wide
deployment.apps/app created
Waiting for deployment "app" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "app" rollout to finish: 1 of 2 updated replicas are available...
deployment "app" successfully rolled out
NAME                      READY   STATUS    RESTARTS       AGE    IP            NODE      NOMINATED NODE   READINESS GATES
api-7779574f48-94phk      1/1     Running   0              83m    10.42.3.41    worker2   <none>           <none>
api-7779574f48-rp4rb      1/1     Running   0              83m    10.42.1.41    worker1   <none>           <none>
api-7779574f48-t86nd      1/1     Running   0              83m    10.42.0.57    master    <none>           <none>
app-5776646bd-5bg9l       1/1     Running   0              2s     10.42.1.42    worker1   <none>           <none>
app-5776646bd-fw6w5       1/1     Running   0              1s     10.42.3.42    worker2   <none>           <none>
broken                    1/1     Running   1 (3h7m ago)   6d4h   10.42.3.35    worker2   <none>           <none>
demo-7b48f66b47-cdp56     1/1     Running   1 (3h7m ago)   6d4h   10.42.1.25    worker1   <none>           <none>
podinfo-f6698f896-j5dz8   1/1     Running   0              164m   10.42.3.36    worker2   <none>           <none>
podinfo-f6698f896-x7q4d   1/1     Running   0              164m   10.42.1.27    worker1   <none>           <none>
podinfo-f6698f896-zvgt2   1/1     Running   0              164m   10.42.0.54    master    <none>           <none>
web-79b7768d68-gv5v9      1/1     Running   0              104m   10.42.3.40    worker2   <none>           <none>
web-79b7768d68-v9bsv      1/1     Running   0              104m   10.42.1.38    worker1   <none>           <none>
```

3.Vérifier que les limites sont bien prises en compte

```
arij@master:~/lab4.2$ kubectl describe pod -l app=app | grep -A6 -E "Limits|Requests" | head -n 14
    Limits:
      cpu:     200m
      memory:  128Mi
    Requests:
      cpu:        100m
      memory:     64Mi
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-wpnzd (ro)
Conditions:
--
    Limits:
      cpu:     200m
      memory:  128Mi
```

4. Voir la consommation réelle vs la réservation

```
arij@master:~/lab4.2$ kubectl top pods
NAME                      CPU(cores)   MEMORY(bytes)
api-7779574f48-94phk      0m           12Mi
api-7779574f48-rp4rb      1m           12Mi
api-7779574f48-t86nd      1m           12Mi
app-5776646bd-5bg9l       0m           2Mi
app-5776646bd-fw6w5       0m           2Mi
broken                    0m           7Mi
demo-7b48f66b47-cdp56     0m           9Mi
podinfo-f6698f896-j5dz8   1m           36Mi
podinfo-f6698f896-x7q4d   1m           36Mi
podinfo-f6698f896-zvgt2   1m           36Mi
web-79b7768d68-gv5v9      0m           2Mi
web-79b7768d68-v9bsv      0m           2Mi
```

5. Retourner voir les réservations sur le nœud

```
arij@master:~/lab4.2$ kubectl describe node worker1 | grep -A8 "Allocated resources"
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests   Limits
  --------           --------   ------
  cpu                100m (5%)  200m (10%)
  memory             64Mi (1%)  128Mi (3%)
  ephemeral-storage  0 (0%)     0 (0%)
  hugepages-1Gi      0 (0%)     0 (0%)
  hugepages-2Mi      0 (0%)     0 (0%)
```

Étape_3 : Diagnostiquer un Pod Pending :

1. Créer le fichier gourmand.yaml

```
arij@master:~/lab4.2$ cat <<'EOF' > gourmand.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gourmand
spec:
  replicas: 1
  selector:
    matchLabels:
      app: gourmand
  template:
    metadata:
      labels:
        app: gourmand
    spec:
      containers:
        - name: app
          image: nginx:1.26-alpine
          resources:
            requests:
              memory: 8Gi
EOF
```

2. Appliquer :

```
arij@master:~/lab4.2$ kubectl apply -f gourmand.yaml
deployment.apps/gourmand created
```

3. observer le blocage

```
arij@master:~/lab4.2$ kubectl get pods -l app=gourmand
NAME                        READY   STATUS    RESTARTS   AGE
gourmand-9784d75c8-dwfbc    0/1     Pending   0          21s
```

4. Lire l'erreur dans les événements

```
arij@master:~/lab4.2$ kubectl describe pod -l app=gourmand | tail -n 6
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  76s   default-scheduler  0/3 nodes are available: 3 Insufficient memory. no new claims to deallocate, pr
eemption: 0/3 nodes are available: 3 Preemption is not helpful for scheduling.
```

5. Réparer le problème :

```
arij@master:~/lab4.2$ sed -i 's|memory: 8Gi|memory: 64Mi|' gourmand.yaml
kubectl apply -f gourmand.yaml
kubectl rollout status deployment/gourmand
kubectl get pods -l app=gourmand
deployment.apps/gourmand configured
Waiting for deployment "gourmand" rollout to finish: 0 out of 1 new replicas have been updated...
Waiting for deployment "gourmand" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "gourmand" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "gourmand" rollout to finish: 1 old replicas are pending termination...
deployment "gourmand" successfully rolled out
NAME                        READY   STATUS    RESTARTS   AGE
gourmand-54dd9bc4db-rt5lx   1/1     Running   0          1s
```

Étape_4 : Provoquer un OOMKilled :

1. Créer le fichier oom.yaml

2. Appliquer

```
arij@master:~/lab4.2$ kubectl apply -f oom.yaml
pod/oom created
```

3. surveiller le pods

```
^Carij@master:~/lab4.2$ kubectl get pod oom -w
NAME   READY   STATUS    RESTARTS        AGE
oom    1/1     Running   1 (2m46s ago)   3m27s
oom    0/1     OOMKilled   1 (3m40s ago)   4m21s
oom    0/1     CrashLoopBackOff   1 (11s ago)     4m31s
oom    1/1     Running            2 (11s ago)     4m31s
```

4. Diagnostiquer la cause exacte

```
^Carij@master:~/lab4.2kubectl describe pod oom | grep -A6 "Last State"e"
    Last State:     Terminated
      Reason:       OOMKilled
      Exit Code:    137
      Started:      Sun, 18 Oct 2026 14:53:35 +0000
      Finished:     Sun, 18 Oct 2026 14:57:13 +0000
    Ready:          True
    Restart Count:  2
```

5. voir les logs :

```
arij@master:~/lab4.2$ kubectl logs oom
arij@master:~/lab4.2$
```

6.suppression :

```
arij@master:~/lab4.2$ kubectl delete pod oom
pod "oom" deleted from tp-k8s namespace
```

Étape_5 : Observer la limite de CP

1. Créer le fichier cpu.yaml

```
arij@master:~/lab4.2$ cat <<'EOF' > cpu.yaml
apiVersion: v1
kind: Pod
metadata:
  name: cpu-libre
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while :; do :; done"]
---
apiVersion: v1
kind: Pod
metadata:
  name: cpu-limite
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while :; do :; done"]
      resources:
        limits:
          cpu: 100m
EOF
```

2.Appliquer et attendre 1 minute :

```
arij@master:~/lab4.2$ kubectl apply -f cpu.yaml
pod/cpu-libre created
pod/cpu-limite created
```

3. Comparer la consommation

```
arij@master:~/lab4.2$ kubectl top pods
NAME                          CPU(cores)   MEMORY(bytes)
api-7779574f48-94phk          1m           12Mi
api-7779574f48-rp4rb          1m           12Mi
api-7779574f48-t86nd          0m           12Mi
app-5776646bd-5bg9l           0m           2Mi
app-5776646bd-fw6w5           0m           2Mi
broken                        0m           7Mi
cpu-libre                     1000m        2Mi
cpu-limite                    101m         0Mi
demo-7b48f66b47-cdp56         0m           9Mi
gourmand-54dd9bc4db-rt5lx     0m           2Mi
podinfo-f6698f896-j5dz8       1m           36Mi
podinfo-f6698f896-x7q4d       0m           36Mi
podinfo-f6698f896-zvgt2       1m           36Mi
web-79b7768d68-gv5v9          0m           2Mi
web-79b7768d68-v9bsv          0m           2Mi
```

4. Nettoyer :

```
arij@master:~/lab4.2$ kubectl delete -f cpu.yaml
pod "cpu-libre" deleted from tp-k8s namespace
pod "cpu-limite" deleted from tp-k8s namespace
arij@master:~/lab4.2$
```

Étape_6 : Poser un ResourceQuota, puis un LimitRange :

1. Créer un nouveau namespace

```
arij@master:~/lab4.2$ kubectl create namespace tp-quota
namespace/tp-quota created
arij@master:~/lab4.2$ kubectl config set-context --current --namespace=tp-quota
Context "default" modified.
```

2. Créer le fichier quota.yaml

```
arij@master:~/lab4.2$ cat <<'EOF' > quota.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: quota
spec:
  hard:
    requests.cpu: 500m
    requests.memory: 512Mi
    limits.cpu: "2"
    limits.memory: 1Gi
    pods: "5"
EOF
```

3. Appliquer le quota

```
arij@master:~/lab4.2$ kubectl apply -f quota.yaml
resourcequota/quota created
```

4. tester la création d'un pod

```
arij@master:~/lab4.2$ kubectl run test --image=nginx:1.26-alpine
Error from server (Forbidden): pods "test" is forbidden: failed quota: quota: must specify limits.cpu for: test; limits.memory for: t
est; requests.cpu for: test; requests.memory for: test
```

5. Créer le fichier limitrange.yaml

```
arij@master:~/lab4.2$ cat <<'EOF' > limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
spec:
  limits:
    - type: Container
      defaultRequest:
        cpu: 50m
        memory: 64Mi
      default:
        cpu: 200m
        memory: 128Mi
EOF
```

6. Appliquer le LimitRange & retenter la création

```
arij@master:~/lab4.2$ kubectl apply -f limitrange.yaml
limitrange/defaults created
arij@master:~/lab4.2$ kubectl run test --image=nginx:1.26-alpine
pod/test created
```

7. Vérifier les ressources

```
arij@master:~/lab4.2$ kubectl get pod test -o jsonpath='{.spec.containers[0].resources}'; echo
{"limits":{"cpu":"200m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"64Mi"}}
arij@master:~/lab4.2$
```

8.Vérifier l'état du quota

```
arij@master:~/lab4.2$ kubectl describe quota quota
Name:           quota
Namespace:      tp-quota
Resource        Used   Hard
--------        ----   ----
limits.cpu      200m   2
limits.memory   128Mi  1Gi
pods            1      5
requests.cpu    50m    500m
requests.memory  64Mi   512Mi
```

Étape_7 : Dépasser le quota :

1. Créer un Deployment :

```
arij@master:~/lab4.2$ kubectl create deployment charge --image=nginx:1.26-alpine --replicas=8
deployment.apps/charge created
arij@master:~/lab4.2$ sleep 10
arij@master:~/lab4.2$ kubectl get deployment charge
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
charge   4/8     4            4           23s
arij@master:~/lab4.2$ kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
charge-6d94dd4887-2s48m   1/1     Running   0          29s
charge-6d94dd4887-8g58f   1/1     Running   0          29s
charge-6d94dd4887-8wlnc   1/1     Running   0          29s
charge-6d94dd4887-m86nc   1/1     Running   0          29s
test                      1/1     Running   0          5m18s
```

2.voir l'erreur dans les événements du ReplicaSet

```
arij@master:~/lab4.2$ kubectl describe replicaset -l app=charge | tail -n 8
  Normal   SuccessfulCreate  69s                replicaset-controller  Created pod: charge-6d94dd4887-8g58f
  Warning  FailedCreate      69s                replicaset-controller  Error creating: pods "charge-6d94dd4887-4smqz" is forbidden: e
xceeded quota: quota, requested: pods=1, used: pods=5, limited: pods=5
  Warning  FailedCreate      69s                replicaset-controller  Error creating: pods "charge-6d94dd4887-7hhrt" is forbidden: e
xceeded quota: quota, requested: pods=1, used: pods=5, limited: pods=5
  Warning  FailedCreate      69s                replicaset-controller  Error creating: pods "charge-6d94dd4887-blgss" is forbidden: e
xceeded quota: quota, requested: pods=1, used: pods=5, limited: pods=5
  Warning  FailedCreate      69s                replicaset-controller  Error creating: pods "charge-6d94dd4887-5zgp2" is forbidden: e
xceeded quota: quota, requested: pods=1, used: pods=5, limited: pods=5
  Warning  FailedCreate      69s                replicaset-controller  Error creating: pods "charge-6d94dd4887-plt4s" is forbidden: e
xceeded quota: quota, requested: pods=1, used: pods=5, limited: pods=5
  Warning  FailedCreate      69s                replicaset-controller  Error creating: pods "charge-6d94dd4887-kcrwq" is forbidden: e
xceeded quota: quota, requested: pods=1, used: pods=5, limited: pods=5
  Warning  FailedCreate      2s (x16 over 69s)  replicaset-controller  (combined from similar events): Error creating: pods "charge-6
d94dd4887-zmgcj" is forbidden: exceeded quota: quota, requested: pods=1, used: pods=5, limited: pods=5
```

3. Tester un pod qui dépasse le quota de CPU

```
arij@master:~/lab4.2$ cat <<'EOF' > gros.yaml
apiVersion: v1
kind: Pod
metadata:
  name: gros
spec:
  containers:
    - name: app
      image: nginx:1.26-alpine
      resources:
        requests:
          cpu: 600m
          memory: 64Mi
        limits:
          cpu: 600m
          memory: 128Mi
EOF
arij@master:~/lab4.2$ kubectl apply -f gros.yaml
Error from server (Forbidden): error when creating "gros.yaml": pods "gros" is forbidden: exceeded quota: quota, requested: pods=1,re
quests.cpu=600m, used: pods=5,requests.cpu=250m, limited: pods=5,requests.cpu=500m
```

4. Revenir à une charge raisonnable

```
arij@master:~/lab4.2$ kubectl apply -f gros.yaml
Error from server (Forbidden): error when creating "gros.yaml": pods "gros" is forbidden: exceeded quota: quota, requested: pods=1,re
quests.cpu=600m, used: pods=5,requests.cpu=250m, limited: pods=5,requests.cpu=500m
arij@master:~/lab4.2$ kubectl scale deployment charge --replicas=2
deployment.apps/charge scaled
arij@master:~/lab4.2$ kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
charge-6d94dd4887-2s48m   1/1     Running   0          2m41s
charge-6d94dd4887-8wlnc   1/1     Running   0          2m41s
test                      1/1     Running   0          7m30s
arij@master:~/lab4.2$ kubectl describe quota quota
Name:           quota
Namespace:      tp-quota
Resource        Used   Hard
--------        ----   ----
limits.cpu      600m   2
limits.memory   384Mi  1Gi
pods            3      5
requests.cpu    150m   500m
requests.memory  192Mi  512Mi
```

Lab 5.1 : Volumes persistants

Étape_1: Préparer le namespace et examiner la StorageClass :

1. Vérifier les classes de stockage disponibles

```
arij@master:~/lab5.1$ kubectl get storageclass
NAME                   PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION   AGE
local-path (default)   rancher.io/local-path   Delete          WaitForFirstConsumer   false                  20d
```

2. Examiner la configuration de local-path

```
arij@master:~/lab5.1$ kubectl get storageclass local-path -o yaml | grep -E "reclaimPolicy|volumeBindingMode|is-default-class"
    storageclass.kubernetes.io/is-default-class: "true"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

Étape_2: Perte de données avec emptyDir

1. Créer le fichier temp.yaml

```
arij@master:~/lab5.1$ cat <<'EOF' > temp.yaml
apiVersion: v1
kind: Pod
metadata:
  name: temp
spec:
  containers:
    - name: app
      image: nginx:1.26-alpine
      volumeMounts:
        - name: donnees
          mountPath: /donnees
  volumes:
    - name: donnees
      emptyDir: {}
EOF
arij@master:~/lab5.1$ kubectl apply -f temp.yaml
pod/temp created
arij@master:~/lab5.1$ kubectl wait --for=condition=Ready pod/temp --timeout=60s
pod/temp condition met
```

2. Écrire des fichiers aux deux endroits

```
arij@master:~/lab5.1$ kubectl exec temp -- sh -c 'echo "dans emptyDir" > /donnees/fichier.txt; echo "dans le conteneur" > /tmp/epheme
re.txt'
kubectl exec temp -- ls /donnees /tmp
/donnees:
fichier.txt

/tmp:
ephemere.txt
```

3. Provoquer un redémarrage du conteneur :

```
arij@master:~/lab5.1$ kubectl exec temp -- nginx -s stop
2026/10/18 15:49:53 [notice] 44#44: signal process started
arij@master:~/lab5.1$ sleep 10
arij@master:~/lab5.1$ kubectl get pod temp
NAME   READY   STATUS    RESTARTS      AGE
temp   1/1     Running   1 (21s ago)   110s
```

4. Vérification :

```
arij@master:~/lab5.1$ kubectl exec temp -- ls /donnees /tmp
/donnees:
fichier.txt

/tmp:
```

5. Supprimer et recréer le Pod

```
arij@master:~/lab5.1$ kubectl delete pod temp
pod "temp" deleted from tp-k8s namespace
arij@master:~/lab5.1$ kubectl apply -f temp.yaml
pod/temp created
arij@master:~/lab5.1$ kubectl wait --for=condition=Ready pod/temp --timeout=60s
pod/temp condition met
arij@master:~/lab5.1$ kubectl exec temp -- ls /donnees
```

Étape_3 : Créer un PVC :

1.Créer le fichier pvc.yaml :

```
arij@master:~/lab5.1$ cat <<'EOF' > pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: local-path
  resources:
    requests:
      storage: 1Gi
EOF
arij@master:~/lab5.1$ kubectl apply -f pvc.yaml
persistentvolumeclaim/data created
```

2. Vérifier l'état du PVC et des PV :

```
arij@master:~/lab5.1$ kubectl get pvc,pv
NAME                         STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data   Pending                                       local-path     <unset>                 34s
```

3. Lire le message d'attente

```
arij@master:~/lab5.1$ kubectl describe pvc data | tail -n 5
Used By:       <none>
Events:
  Type    Reason                Age                From                         Message
  ----    ------                ----               ----                         -------
  Normal  WaitForFirstConsumer  13s (x5 over 62s)  persistentvolume-controller  waiting for first consumer to be created before bindi
ng
```

Étape_4 : Utiliser le PVC dans un Pod :

1.Créer le fichier app.yaml

```
arij@master:~/lab5.1$ cat <<'EOF' > app.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
    - name: app
      image: nginx:1.26-alpine
      volumeMounts:
        - name: donnees
          mountPath: /usr/share/nginx/html
  volumes:
    - name: donnees
      persistentVolumeClaim:
        claimName: data
EOF
arij@master:~/lab5.1$ kubectl apply -f app.yaml
pod/app created
```

2.Observer le passage de Pending à Bound

```
arij@master:~/lab5.1$ kubectl get pvc -w
NAME   STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data   Bound    pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36   1Gi        RWO            local-path     <unset>                 4m30s
```

3. Vérifier le PV et l'affinité du nœud

```
^Carij@master:~/lab5.1kubectl get pvc,pvpv
NAME                         STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRI
BUTESCLASS   AGE
persistentvolumeclaim/data   Bound    pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36   1Gi        RWO            local-path     <unset>
             6m17s

NAME                                                        CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM          STORAGE
CLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36   1Gi        RWO            Delete           Bound    tp-k8s/data    local-p
ath     <unset>                          2m29s
arij@master:~/lab5.1$ kubectl get pod app -o wide
NAME   READY   STATUS    RESTARTS   AGE     IP           NODE      NOMINATED NODE   READINESS GATES
app    1/1     Running   0          2m42s   10.42.3.49   worker2   <none>           <none>
arij@master:~/lab5.1$ PV=$(kubectl get pvc data -o jsonpath='{.spec.volumeName}')
arij@master:~/lab5.1$ kubectl describe pv $PV | grep -E -A6 "Node Affinity|Source"
Node Affinity:
  Required Terms:
    Term 0:        kubernetes.io/hostname in [worker2]
Message:
Source:
    Type:  LocalVolume (a persistent volume backed by local storage on a node)
    Path:  /var/lib/rancher/k3s/storage/pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36_tp-k8s_data
Events:    <none>
```

4.Écrire une page web et tester :

```
arij@master:~/lab5.1$ kubectl exec app -- sh -c 'echo "Bonjour, donnee persistante" > /usr/share/nginx/html/index.html'
arij@master:~/lab5.1$ kubectl exec app -- wget -qO- http://localhost
Bonjour, donnee persistante
```

5. Vérifier sur le disque dur du nœud

```
arij@worker2:~$ sudo ls -R /var/lib/rancher/k3s/storage/
[sudo] password for arij:
/var/lib/rancher/k3s/storage/:
pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36_tp-k8s_data

/var/lib/rancher/k3s/storage/pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36_tp-k8s_data:
index.html
```

Étape_5 : Prouver la persistance

1. Supprimer le Pod

```
arij@master:~/lab5.1$ kubectl delete pod app
kubectl get pvc
pod "app" deleted from tp-k8s namespace
NAME   STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data   Bound    pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36   1Gi        RWO            local-path     <unset>                 11m
```

2. Recréer le Pod :

```
arij@master:~/lab5.1$ kubectl apply -f app.yaml
pod/app created
arij@master:~/lab5.1$ kubectl wait --for=condition=Ready pod/app --timeout=60s
pod/app condition met
arij@master:~/lab5.1$ kubectl get pod app -o wide
NAME   READY   STATUS    RESTARTS   AGE   IP           NODE      NOMINATED NODE   READINESS GATES
app    1/1     Running   0          11s   10.42.3.50   worker2   <none>           <none>
```

3. Vérifier :

```
arij@master:~/lab5.1$ kubectl exec app -- wget -qO- http://localhost
Bonjour, donnee persistante
arij@master:~/lab5.1$
```

Étape_6 : Supprimer le PVC :

1. suppression du PVC

```
arij@master:~/lab5.1$ kubectl delete pvc data --wait=false
persistentvolumeclaim "data" deleted from tp-k8s namespace
arij@master:~/lab5.1$ kubectl get pvc
NAME   STATUS        VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data   Terminating   pvc-7a5ff5ed-88c8-4a35-bdcb-316b9d17ce36   1Gi        RWO            local-path     <unset>                 15m
```

2. Supprimer le Pod pour débloquer la suppression

```
arij@master:~/lab5.1$ kubectl delete pod app
pod "app" deleted from tp-k8s namespace
arij@master:~/lab5.1$ sleep 10
arij@master:~/lab5.1$ kubectl get pvc,pv
No resources found
```

3. Vérifier sur le disque dur de worker2

```
arij@worker2:~$ sudo ls -R /var/lib/rancher/k3s/storage/
/var/lib/rancher/k3s/storage/:
arij@worker2:~$
```

4. Recréer le PVC et le Pod :

```
arij@master:~/lab5.1$ kubectl apply -f pvc.yaml
persistentvolumeclaim/data created
arij@master:~/lab5.1$ kubectl apply -f app.yaml
pod/app created
arij@master:~/lab5.1$ kubectl wait --for=condition=Ready pod/app --timeout=60s
pod/app condition met
arij@master:~/lab5.1$ kubectl exec app -- ls /usr/share/nginx/html
arij@master:~/lab5.1$
```

Étape_7 : Diagnostiquer un PVC en Pending :

1. Créer le fichier pvc-erreur.yaml

```
arij@master:~/lab5.1$ cat <<'EOF' > pvc-erreur.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-erreur
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: local-pat
  resources:
    requests:
      storage: 1Gi
EOF
arij@master:~/lab5.1$ kubectl apply -f pvc-erreur.yaml
persistentvolumeclaim/data-erreur created
```

2. Diagnostiquer le blocage :

```
arij@master:~/lab5.1$ kubectl get pvc data-erreur
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data-erreur   Pending                                      local-pat      <unset>                 43s
arij@master:~/lab5.1$ kubectl describe pvc data-erreur | tail -n 5
Used By:       <none>
Events:
  Type     Reason              Age              From                         Message
  ----     ------              ----             ----                         -------
  Warning  ProvisioningFailed  7s (x4 over 49s)  persistentvolume-controller  storageclass.storage.k8s.io "local-pat" not found
```

3. Réparer l'erreur

* *supprime le PVC en erreur*

```
arij@master:~/lab5.1$ kubectl delete pvc data-erreur
persistentvolumeclaim "data-erreur" deleted from tp-k8s namespace
```

* *corrige la faute de frappe dans le fichier*

```
arij@master:~/lab5.1$ sed -i 's|local-pat$|local-path|' pvc-erreur.yaml
arij@master:~/lab5.1$
```

* *recrée le PVC*

```
arij@master:~/lab5.1$ kubectl apply -f pvc-erreur.yaml
persistentvolumeclaim/data-erreur created
arij@master:~/lab5.1$ kubectl get pvc data-erreur
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data-erreur   Pending                                      local-path     <unset>                 6s
arij@master:~/lab5.1$
```

Lab 5.2 : Base de données avec StatefulSet

Étape_1 : Préparer la configuration :

1. Créer le fichier app-config.yaml

```
arij@master:~/lab5.2$ cat <<'EOF' > app-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_HOST: db
  DB_PORT: "5432"
  DB_NAME: appdb
EOF
```

2. Créer le Secret

```
arij@master:~/lab5.2$ kubectl delete secret db-credentials
kubectl create secret generic db-credentials \
  --from-literal=POSTGRES_USER=appuser \
  --from-literal=POSTGRES_PASSWORD='ChangeMe-123'
secret "db-credentials" deleted from tp-k8s namespace
secret/db-credentials created
```

3. Vérifier

```
arij@master:~/lab5.2$ kubectl get configmap,secret
NAME                         DATA   AGE
configmap/app-config         3      6d7h
configmap/config-manquant    1      6d6h
configmap/kube-root-ca.crt   1      6d7h
configmap/nginx-conf         1      6d6h

NAME                     TYPE     DATA   AGE
secret/db-credentials    Opaque   2      2m7s
```

Étape_2 : Déployer la base avec un StatefulSet :

1. Créez le Service headless et le StatefulSet

```
arij@master:~/lab5.2$ cat <<'EOF' > db.yaml
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  clusterIP: None              # Service headless
  selector:
    app: appdb
    tier: db
  ports:
    - port: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db              # nom du Service headless
  replicas: 1
  selector:
    matchLabels:
      app: appdb
      tier: db
  template:
    metadata:
      labels:
        app: appdb
        tier: db
    spec:
      containers:
        - name: postgres
```

```
arij@master:~/lab5.2$ kubectl apply -f db.yaml
service/db created
statefulset.apps/db created
arij@master:~/lab5.2$
```

2. Observer la création du pod db-0

```
arij@master:~/lab5.2$ kubectl get pods -w
NAME                      READY   STATUS              RESTARTS        AGE
api-7779574f48-94phk      1/1     Running             0               4h3m
api-7779574f48-rp4rb      1/1     Running             0               4h3m
api-7779574f48-t86nd      1/1     Running             0               4h3m
app                       1/1     Running             0               63m
broken                    1/1     Running             1 (5h48m ago)   6d6h
db-0                      0/1     ContainerCreating   0               57s
demo-7b48f66b47-cdp56     1/1     Running             1 (5h48m ago)   6d6h
podinfo-f6698f896-j5dz8   1/1     Running             0               5h25m
podinfo-f6698f896-x7q4d   1/1     Running             0               5h25m
podinfo-f6698f896-zvgt2   1/1     Running             0               5h25m
temp                      1/1     Running             0               92m
web-79b7768d68-gv5v9      1/1     Running             0               4h25m
web-79b7768d68-v9bsv      1/1     Running             0               4h25m
db-0                      1/1     Running             0               59s
```

3. Vérifier le StatefulSet & le stockage

```
^Carij@master:~/lab5.2kubectl get statefulset,service,pvc,pvpv
NAME                 READY   AGE
statefulset.apps/db  1/1     2m14s

NAME                       TYPE           CLUSTER-IP      EXTERNAL-IP                                    PORT(S)          AGE
service/api-svc            ClusterIP      10.43.20.42     <none>                                         80/TCP           4h26m
service/db                 ClusterIP      None            <none>                                         5432/TCP         2m14s
service/podinfo-lb         LoadBalancer   10.43.67.255    192.168.159.10,192.168.159.11,192.168.159.12   8080:31928/TCP   4h54m
service/podinfo-nodeport   NodePort       10.43.150.59    <none>                                         80:32284/TCP     5h
service/podinfo-svc        ClusterIP      10.43.186.72    <none>                                         80/TCP           5h22m
service/web-svc            ClusterIP      10.43.71.54     <none>                                         80/TCP           6d6h

NAME                                    STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOL
UMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data              Bound     pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            local-path     <un
set>                 64m
persistentvolumeclaim/data-db-0         Bound     pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            local-path     <un
set>                 2m13s
persistentvolumeclaim/data-erreur       Pending                                                                             local-path     <un
set>                 56m

NAME                                                CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM              ST
ORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            Delete           Bound    tp-k8s/data-db-0   lo
cal-path     <unset>                          2m1s
persistentvolume/pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            Delete           Bound    tp-k8s/data        lo
cal-path     <unset>                          64m
arij@master:~/lab5.2$ kubectl logs db-0 | tail -n 5
2026-10-18 17:24:04.841 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2026-10-18 17:24:04.841 UTC [1] LOG:  listening on IPv6 address "::", port 5432
2026-10-18 17:24:04.850 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-10-18 17:24:04.875 UTC [57] LOG:  database system was shut down at 2026-10-18 17:24:04 UTC
2026-10-18 17:24:04.988 UTC [1] LOG:  database system is ready to accept connections
```

```
arij@master:~/lab5.2$ kubectl exec db-0 -- pg_isready -U appuser -d appdb
/var/run/postgresql:5432 - accepting connections
```

Étape_3 : Créer des données :

1. Créer la table

```
arij@master:~/lab5.2$ kubectl exec db-0 -- psql -U appuser -d appdb -c "CREATE TABLE taches (id serial PRIMARY KEY, titre text NOT NU
LL, faite boolean DEFAULT false);"
CREATE TABLE
```

2. Insérer des données

```
arij@master:~/lab5.2$ kubectl exec db-0 -- psql -U appuser -d appdb -c "INSERT INTO taches (titre) VALUES ('Apprendre les StatefulSet
s'), ('Tester la persistance');"
INSERT 0 2
```

3. Vérifier le contenu

```
arij@master:~/lab5.2$ kubectl exec db-0 -- psql -U appuser -d appdb -c "INSERT INTO taches (titre) VALUES ('Apprendre les StatefulSet
s'), ('Tester la persistance');"
INSERT 0 2
```

Étape_4 : Prouver la persistance :

1. Supprimer le pod & Surveiller la recréation

```
arij@master:~/lab5.2$ kubectl get pod db-0 -o wide
NAME   READY   STATUS    RESTARTS   AGE     IP           NODE      NOMINATED NODE   READINESS GATES
db-0   1/1     Running   0          6m59s   10.42.1.49   worker1   <none>           <none>
arij@master:~/lab5.2$ kubectl delete pod db-0
pod "db-0" deleted from tp-k8s namespace
arij@master:~/lab5.2$ kubectl get pods -w
NAME                      READY   STATUS    RESTARTS        AGE
api-7779574f48-94phk      1/1     Running   0               4h9m
api-7779574f48-rp4rb      1/1     Running   0               4h9m
api-7779574f48-t86nd      1/1     Running   0               4h9m
app                       1/1     Running   0               69m
broken                    1/1     Running   1 (5h54m ago)   6d6h
db-0                      1/1     Running   0               5s
demo-7b48f66b47-cdp56     1/1     Running   1 (5h54m ago)   6d7h
podinfo-f6698f896-j5dz8   1/1     Running   0               5h31m
podinfo-f6698f896-x7q4d   1/1     Running   0               5h31m
podinfo-f6698f896-zvgt2   1/1     Running   0               5h31m
temp                      1/1     Running   0               98m
web-79b7768d68-gv5v9      1/1     Running   0               4h31m
web-79b7768d68-v9bsv      1/1     Running   0               4h31m
db-0                      1/1     Running   0               84s
```

2. Vérifier que les données ont survécu

```
^Carij@master:~/lab5.2kubectl exec db-0 -- pg_isready -U appuser -d appdbdb
/var/run/postgresql:5432 - accepting connections
arij@master:~/lab5.2$ kubectl exec db-0 -- psql -U appuser -d appdb -c "SELECT * FROM taches;"
 id |           titre            | faite
----+----------------------------+-------
  1 | Apprendre les StatefulSets | f
  2 | Tester la persistance      | f
  3 | Apprendre les StatefulSets | f
  4 | Tester la persistance      | f
(4 rows)
```

```
arij@master:~/lab5.2$ kubectl get pod db-0 -o wide
NAME   READY   STATUS    RESTARTS   AGE     IP           NODE      NOMINATED NODE   READINESS GATES
db-0   1/1     Running   0          3m50s   10.42.1.50   worker1   <none>           <none>
arij@master:~/lab5.2$ kubectl get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   A
GE
data          Bound     pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            local-path     <unset>                 7
3m
data-db-0     Bound     pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            local-path     <unset>                 1
1m
data-erreur   Pending                                              local-path     <unset>                 6
5m
```

Étape_5 : Joindre la base par son nom DNS :

1. Tester le DNS avec un pod temporaire

```
arij@master:~/lab5.2$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- nslookup db
Server:         10.43.0.10
Address:        10.43.0.10:53

Name:   db.tp-k8s.svc.cluster.local
Address: 10.42.1.50

** server can't find db.cluster.local: NXDOMAIN

** server can't find db.cluster.local: NXDOMAIN

** server can't find db.svc.cluster.local: NXDOMAIN

** server can't find db.svc.cluster.local: NXDOMAIN


All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
warning: couldn't attach to pod/test, falling back to streaming logs: unable to upgrade connection: container test not found in pod t
est_tp-k8s
Server:         10.43.0.10
Address:        10.43.0.10:53

Name:   db.tp-k8s.svc.cluster.local
Address: 10.42.1.50

** server can't find db.cluster.local: NXDOMAIN

** server can't find db.cluster.local: NXDOMAIN

** server can't find db.svc.cluster.local: NXDOMAIN
```

```
arij@master:~/lab5.2$ kubectl run test --rm -it --image=busybox:1.36 --restart=Never -- nslookup db-0.db
Server:         10.43.0.10
Address:        10.43.0.10:53

** server can't find db-0.db: NXDOMAIN

** server can't find db-0.db: NXDOMAIN

All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
Session ended, resume using 'kubectl attach test -c test -n tp-k8s -i -t' command
pod "test" deleted from tp-k8s namespace
pod tp-k8s/test terminated (Error)
```

2. Se connecter avec un client PostgreSQL

```
arij@master:~/lab5.2$ kubectl run client --rm -it --image=postgres:16-alpine --restart=Never \
  --env="PGPASSWORD=ChangeMe-123" -- \
  psql -h db -U appuser -d appdb -c "SELECT count(*) FROM taches;"
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
 count
-------
     4
(1 row)

Session ended, resume using 'kubectl attach client -c client -n tp-k8s -i -t' command
pod "client" deleted from tp-k8s namespace
```

Étape_6 : Passer à deux réplicas :

1. Augmenter à 2 réplicas

```
arij@master:~/lab5.2$ kubectl scale statefulset db --replicas=2
statefulset.apps/db scaled
```

```
arij@master:~/lab5.2$ kubectl scale statefulset db --replicas=2
statefulset.apps/db scaled
arij@master:~/lab5.2$ kubectl get pods -w
NAME                      READY   STATUS    RESTARTS       AGE
api-7779574f48-94phk      1/1     Running   0              4h18m
api-7779574f48-rp4rb      1/1     Running   0              4h18m
api-7779574f48-t86nd      1/1     Running   0              4h18m
app                       1/1     Running   0              77m
broken                    1/1     Running   1 (6h3m ago)   6d7h
db-0                      1/1     Running   0              8m42s
db-1                      1/1     Running   0              7s
demo-7b48f66b47-cdp56     1/1     Running   1 (6h3m ago)   6d7h
podinfo-f6698f896-j5dz8   1/1     Running   0              5h40m
podinfo-f6698f896-x7q4d   1/1     Running   0              5h40m
podinfo-f6698f896-zvgt2   1/1     Running   0              5h40m
temp                      1/1     Running   0              107m
web-79b7768d68-gv5v9      1/1     Running   0              4h39m
web-79b7768d68-v9bsv      1/1     Running   0              4h39m
db-1                      1/1     Running   0              93s
```

2. Comparer les deux bases de données

```
arij@master:~/lab5.2$ kubectl get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   A
GE
data          Bound     pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            local-path     <unset>                 8
0m
data-db-0     Bound     pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            local-path     <unset>                 1
8m
data-db-1     Bound     pvc-1e8a791d-560f-4e20-9f65-d960a9af43a0   1Gi        RWO            local-path     <unset>                 2
m57s
data-erreur   Pending                                                                        local-path     <unset>                 7
2m
arij@master:~/lab5.2$ kubectl exec db-1 -- psql -U appuser -d appdb -c "\dt"
Did not find any relations.
arij@master:~/lab5.2$ kubectl exec db-1 -- psql -U appuser -d appdb -c "\dt"
Did not find any relations.
arij@master:~/lab5.2$ kubectl exec db-0 -- psql -U appuser -d appdb -c "\dt"
         List of relations
 Schema |  Name  | Type  |  Owner
--------+--------+-------+---------
 public | taches | table | appuser
(1 row)
```

3. Revenir à 1 réplica et nettoyer

```
arij@master:~/lab5.2$ kubectl scale statefulset db --replicas=1
statefulset.apps/db scaled
arij@master:~/lab5.2$ kubectl get pods
NAME                      READY   STATUS    RESTARTS       AGE
api-7779574f48-94phk      1/1     Running   0              4h24m
api-7779574f48-rp4rb      1/1     Running   0              4h24m
api-7779574f48-t86nd      1/1     Running   0              4h24m
app                       1/1     Running   0              84m
broken                    1/1     Running   1 (6h9m ago)   6d7h
db-0                      1/1     Running   0              15m
demo-7b48f66b47-cdp56     1/1     Running   1 (6h9m ago)   6d7h
podinfo-f6698f896-j5dz8   1/1     Running   0              5h46m
podinfo-f6698f896-x7q4d   1/1     Running   0              5h46m
podinfo-f6698f896-zvgt2   1/1     Running   0              5h46m
temp                      1/1     Running   0              113m
web-79b7768d68-gv5v9      1/1     Running   0              4h46m
web-79b7768d68-v9bsv      1/1     Running   0              4h46m
arij@master:~/lab5.2$ kubectl get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   A
GE
data          Bound     pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            local-path     <unset>                 8
4m
data-db-0     Bound     pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            local-path     <unset>                 2
2m
data-db-1     Bound     pvc-1e8a791d-560f-4e20-9f65-d960a9af43a0   1Gi        RWO            local-path     <unset>                 6
m37s
data-erreur   Pending                                                                        local-path     <unset>                 7
6m
```

4. supprimer manuellement

```
arij@master:~/lab5.2$ kubectl delete pvc data-db-1
persistentvolumeclaim "data-db-1" deleted from tp-k8s namespace
arij@master:~/lab5.2$ kubectl get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   A
GE
data          Bound     pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            local-path     <unset>                 8
5m
data-db-0     Bound     pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            local-path     <unset>                 2
3m
data-erreur   Pending                                                                        local-path     <unset>                 7
7m
```

Étape_7 : Comprendre le rôle du mot de passe initial

1. Mettre à jour le Secret

```
arij@master:~/lab5.2$ kubectl create secret generic db-credentials \
  --from-literal=POSTGRES_USER=appuser \
  --from-literal=POSTGRES_PASSWORD='Nouveau-456' \
  --dry-run=client -o yaml | kubectl apply -f -
Warning: resource secrets/db-credentials is missing the kubectl.kubernetes.io/last-applied-configuration annotation which is required
 by kubectl apply. kubectl apply should only be used on resources created declaratively by either kubectl create --save-config or kub
ectl apply. The missing annotation will be patched automatically.
secret/db-credentials configured
```

2. Redémarrer la base de données

```
arij@master:~/lab5.2$ kubectl rollout restart statefulset db
kubectl rollout status statefulset db
kubectl exec db-0 -- pg_isready -U appuser -d appdb
statefulset.apps/db restarted
Waiting for partitioned roll out to finish: 0 out of 1 new pods have been updated...
Waiting for 1 pods to be ready...
Waiting for 1 pods to be ready...
Waiting for 1 pods to be ready...
partitioned roll out complete: 1 new pods have been updated...
/var/run/postgresql:5432 - accepting connections
```

3. Tester le nouveau mot de passe

```
arij@master:~/lab5.2$ kubectl run client --rm -it --image=postgres:16-alpine --restart=Never \
  --env="PGPASSWORD=Nouveau-456" -- \
  psql -h db -U appuser -d appdb -c "SELECT 1;"
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
Session ended, resume using 'kubectl attach client -c client -n tp-k8s -i -t' command
pod "client" deleted from tp-k8s namespace
pod tp-k8s/client terminated (Error)
arij@master:~/lab5.2$
```

4. Tester l'ancien mot de passe

```
arij@master:~/lab5.2$ kubectl run client --rm -it --image=postgres:16-alpine --restart=Never \
  --env="PGPASSWORD=ChangeMe-123" -- \
  psql -h db -U appuser -d appdb -c "SELECT 1;"
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
Session ended, resume using 'kubectl attach client -c client -n tp-k8s -i -t' command
pod "client" deleted from tp-k8s namespace
arij@master:~/lab5.2$
```

Lab 6.1 : Jobs et CronJobs

Étape_1 : Préparer le namespace :

```
arij@master:~$ mkdir lab6.1
arij@master:~$ cd lab6.1
arij@master:~/lab6.1$ kubectl config set-context --current --namespace=tp-k8s
Context "default" modified.
arij@master:~/lab6.1$
```

Étape_2 : Un Job simple :

1. Créer le fichier calcul.yaml

```
arij@master:~/lab6.1$ cat <<'EOF' > calcul.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: calcul
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: calcul
          image: busybox:1.36
          command: ["sh", "-c", "echo debut; sleep 5; echo fin"]
EOF
arij@master:~/lab6.1$ kubectl apply -f calcul.yaml
job.batch/calcul created
```

2. vérifier

```
arij@master:~/lab6.1$ kubectl wait --for=condition=complete job/calcul --timeout=60s
job.batch/calcul condition met
arij@master:~/lab6.1$ kubectl get job,pods
NAME              STATUS     COMPLETIONS   DURATION   AGE
job.batch/calcul   Complete   1/1           9s         25s

NAME                          READY   STATUS      RESTARTS        AGE
pod/api-7779574f48-94phk      1/1     Running     0               4h44m
pod/api-7779574f48-rp4rb      1/1     Running     0               4h44m
pod/api-7779574f48-t86nd      1/1     Running     0               4h44m
pod/app                       1/1     Running     0               103m
pod/broken                    1/1     Running     1 (6h28m ago)   6d7h
pod/calcul-jkpzb              0/1     Completed   0               24s
pod/db-0                      1/1     Running     0               15m
pod/demo-7b48f66b47-cdp56     1/1     Running     1 (6h28m ago)   6d7h
pod/podinfo-f6698f896-j5dz8   1/1     Running     0               6h5m
pod/podinfo-f6698f896-x7q4d   1/1     Running     0               6h5m
pod/podinfo-f6698f896-zvgt2   1/1     Running     0               6h5m
pod/temp                      1/1     Running     0               132m
pod/web-79b7768d68-gv5v9      1/1     Running     0               5h5m
pod/web-79b7768d68-v9bsv      1/1     Running     0               5h5m
arij@master:~/lab6.1$ kubectl logs job/calcul
debut
fin
arij@master:~/lab6.1$ kubectl describe job calcul | grep -E "Pods Statuses|Completions"
Completions:      1
Pods Statuses:    0 Active (0 Ready) / 1 Succeeded / 0 Failed
```

Étape_3 : Un Job en échec :

1.Créer le fichier echec.yaml

```
arij@master:~/lab6.1$ cat <<'EOF' > echec.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: echec
spec:
  backoffLimit: 2
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: app
          image: busybox:1.36
          command: ["sh", "-c", "echo tentative; exit 1"]
EOF
```

```
^Carij@master:~/lab6.1kubectl apply -f echec.yamlml
job.batch/echec created
```

2.Observer les tentatives

```
NAME          READY   STATUS   RESTARTS   AGE
echec-nhqng   0/1     Error    0          4s
echec-nhqng   0/1     Error    0          4s
echec-cmmxc   0/1     Pending   0          0s
echec-cmmxc   0/1     Pending   0          0s
echec-cmmxc   0/1     ContainerCreating   0          0s
echec-cmmxc   0/1     ContainerCreating   0          1s
echec-cmmxc   1/1     Running             0          1s
```

3. Diagnostiquer l'échec

```
^Carij@master:~/lab6.1kubectl get job echecec

NAME    STATUS   COMPLETIONS   DURATION   AGE
echec   Failed   0/1           2m4s       2m4s
arij@master:~/lab6.1$
arij@master:~/lab6.1$ kubectl describe job echec | tail -n 8
  Tolerations:     <none>
Events:
  Type     Reason                Age   From            Message
  ----     ------                ----  ----            -------
  Normal   SuccessfulCreate      2m8s  job-controller  Created pod: echec-nhqng
  Normal   SuccessfulCreate      116s  job-controller  Created pod: echec-cmmxc
  Normal   SuccessfulCreate      95s   job-controller  Created pod: echec-2s76p
  Warning  BackoffLimitExceeded  91s   job-controller  Job has reached the specified backoff limit
arij@master:~/lab6.1$ kubectl logs -l job-name=echec
tentative
tentative
tentative
```

4. Tester la politique OnFailure

```
arij@master:~/lab6.1$ sed -e 's/name: echec/name: echec-onfailure/' -e 's/restartPolicy: Never/restartPolicy: OnFailure/' echec.yaml
| kubectl apply -f -
job.batch/echec-onfailure created
arij@master:~/lab6.1$ sleep 40
arij@master:~/lab6.1$ kubectl get pods -l job-name=echec-onfailure
No resources found in tp-k8s namespace.
```

Étape_4 : Un Job parallèle avec nettoyage automatique :

1. Créer le fichier parallele.yaml

```
arij@master:~/lab6.1$ cat <<'EOF' > parallele.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: parallele
spec:
  completions: 6
  parallelism: 2
  ttlSecondsAfterFinished: 120
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: tache
          image: busybox:1.36
          command: ["sh", "-c", "echo traitement; sleep 10"]
EOF
```

2.Appliquer & observer :

```
arij@master:~/lab6.1$ kubectl apply -f parallele.yaml
job.batch/parallele created
arij@master:~/lab6.1$ kubectl get pods -l job-name=parallele -w
NAME              READY   STATUS    RESTARTS   AGE
parallele-5jb7d   1/1     Running   0          7s
parallele-rw55q   1/1     Running   0          7s
parallele-rw55q   0/1     Completed   0          11s
parallele-5jb7d   0/1     Completed   0          12s
parallele-rw55q   0/1     Completed   0          13s
parallele-5jb7d   0/1     Completed   0          13s
parallele-g7stg   0/1     Pending     0          0s
parallele-lw7r9   0/1     Pending     0          0s
parallele-g7stg   0/1     Pending     0          0s
parallele-lw7r9   0/1     Pending     0          1s
parallele-rw55q   0/1     Completed   0          14s
parallele-5jb7d   0/1     Completed   0          14s
```

Exécution dans Terminal 2 :

```
arij@master:~/lab6.1$ kubectl get pods -l job-name=parallele -w
NAME              READY   STATUS      RESTARTS   AGE
parallele-5jb7d   0/1     Completed   0          43s
parallele-dfzb2   0/1     Completed   0          16s
parallele-g7stg   0/1     Completed   0          30s
parallele-lw7r9   0/1     Completed   0          30s
parallele-rbld5   0/1     Completed   0          16s
parallele-rw55q   0/1     Completed   0          43s
```

Étape_5 : Un CronJob

1. Créer le fichier sauvegarde.yaml

```
arij@master:~/lab6.1$ cat <<'EOF' > sauvegarde.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: sauvegarde
spec:
  schedule: "*/1 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: sauvegarde
              image: busybox:1.36
              command: ["sh", "-c", "date; echo sauvegarde effectuee"]
EOF
arij@master:~/lab6.1$ kubectl apply -f sauvegarde.yaml
cronjob.batch/sauvegarde created
arij@master:~/lab6.1$ kubectl get cronjob
NAME         SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sauvegarde   */1 * * * *   <none>     False     0        <none>          11s
```

2 .Déclencher un Job manuellement :

```
arij@master:~/lab6.1$ kubectl create job --from=cronjob/sauvegarde test-manuel
job.batch/test-manuel created
arij@master:~/lab6.1$ kubectl wait --for=condition=complete job/test-manuel --timeout=60s
job.batch/test-manuel condition met
arij@master:~/lab6.1$ kubectl logs job/test-manuel
Sun Oct 18 18:42:21 UTC 2026
sauvegarde effectuee
```

3. Observer la planification

```
arij@master:~/lab6.1$ sleep 120
arij@master:~/lab6.1$ kubectl get cronjob
NAME         SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sauvegarde   */1 * * * *   <none>     False     1        1s              11m
arij@master:~/lab6.1$ kubectl get jobs
NAME                  STATUS     COMPLETIONS   DURATION   AGE
calcul                Complete   1/1           9s         49m
echec                 Failed     0/1           24m        24m
echec-onfailure       Failed     0/1           21m        21m
sauvegarde-29872491   Complete   1/1           5s         2m6s
sauvegarde-29872492   Complete   1/1           4s         66s
sauvegarde-29872493   Complete   1/1           4s         6s
```

4.Lire les logs du dernier Job créé automatiquement

```
arij@master:~/lab6.1$ JOB=$(kubectl get jobs --sort-by=.metadata.creationTimestamp -o name | tail -n 1)
arij@master:~/lab6.1$ echo "Le dernier Job est : $JOB"
Le dernier Job est : job.batch/sauvegarde-29872493
arij@master:~/lab6.1$ kubectl logs $JOB
Sun Oct 18 18:53:01 UTC 2026
sauvegarde effectuee
```

Étape_6 : Concurrence et suspension :

1. Créer un CronJob "lent"

```
arij@master:~/lab6.1$ cat <<'EOF' > lent.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: lent
spec:
  schedule: "*/1 * * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: lent
              image: busybox:1.36
              command: ["sh", "-c", "echo debut; sleep 150; echo fin"]
EOF
arij@master:~/lab6.1$ kubectl apply -f lent.yaml
cronjob.batch/lent created
arij@master:~/lab6.1$
```

2. Observer le résultat

```
arij@master:~/lab6.1$ sleep 150
arij@master:~/lab6.1$ kubectl get jobs | grep lent
lent-29872497         Complete   1/1           2m34s      3m3s
lent-29872499         Running    0/1           29s        29s
arij@master:~/lab6.1$ kubectl describe cronjob lent | tail -n 6
  Type     Reason            Age                From                Message
  ----     ------            ----               ----                -------
  Normal   SuccessfulCreate  3m9s               cronjob-controller  Created job lent-29872497
  Normal   SawCompletedJob   35s                cronjob-controller  Saw completed job: lent-29872497, condition: Complete
  Normal   SuccessfulCreate  35s                cronjob-controller  Created job lent-29872499
  Normal   JobAlreadyActive  9s (x5 over 2m9s)  cronjob-controller  Not starting job because prior execution is running and concurrenc
y policy is Forbid
```

3. Supprimer le CronJob lent et suspendre sauvegarde

```
arij@master:~/lab6.1$ kubectl delete cronjob lent
cronjob.batch "lent" deleted from tp-k8s namespace
arij@master:~/lab6.1$ kubectl patch cronjob sauvegarde -p '{"spec":{"suspend":true}}'
cronjob.batch/sauvegarde patched
arij@master:~/lab6.1$ kubectl get cronjob
NAME          SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sauvegarde    */1 * * * *   <none>     True      0        61s             19m
```

4. Vérifier qu'aucun nouveau job n'est créé

```
arij@master:~/lab6.1$ sleep 90
arij@master:~/lab6.1$ kubectl get jobs
NAME                  STATUS     COMPLETIONS   DURATION   AGE
calcul                Complete   1/1           9s         80m
echec                 Failed     0/1           55m        55m
echec-onfailure       Failed     0/1           52m        52m
sauvegarde-29872498   Complete   1/1           3s         26m
sauvegarde-29872499   Complete   1/1           3s         25m
sauvegarde-29872500   Complete   1/1           4s         24m
```

5. Réactiver le CronJob

```
arij@master:~/lab6.1$ kubectl patch cronjob sauvegarde -p '{"spec":{"suspend":false}}'
kubectl get cronjob
cronjob.batch/sauvegarde patched
NAME          SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sauvegarde    */1 * * * *   <none>     False     1        11s             42m
```

Étape_7 : Sauvegarde sur un volume persistant

1. Supprimer l'ancien CronJob et créer le nouveau avec le volume

```
arij@master:~/lab6.1$ cat <<'EOF' > sauvegarde-vol.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: backup-pvc
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: local-path
  resources:
    requests:
      storage: 1Gi
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: sauvegarde-vol
spec:
  schedule: "*/1 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: sauvegarde
              image: busybox:1.36
              command: ["sh", "-c", "echo \"sauvegarde du $(date)\" > /backup/dump-$(date +%Y%m%d-%H%M%S).txt"]
              volumeMounts:
                - name: backup
                  mountPath: /backup
          volumes:
```

```
arij@master:~/lab6.1$ kubectl apply -f sauvegarde-vol.yaml
persistentvolumeclaim/backup-pvc created
cronjob.batch/sauvegarde-vol created
arij@master:~/lab6.1$ kubectl get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   A
GE
backup-pvc    Pending                                                                        local-path     <unset>                 6
s
data          Bound     pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            local-path     <unset>                 3
h8m
data-db-0     Bound     pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            local-path     <unset>                 1
26m
data-erreur   Pending                                                                        local-path     <unset>                 3
h
```

2. Attendre 3 minute & Observation

```
arij@master:~/lab6.1$ sleep 180
arij@master:~/lab6.1$ ^C
arij@master:~/lab6.1$ kubectl get jobs
NAME                     STATUS     COMPLETIONS   DURATION   AGE
calcul                   Complete   1/1           9s         96m
echec                    Failed     0/1           71m        71m
echec-onfailure          Failed     0/1           68m        68m
sauvegarde-29872534      Complete   1/1           3s         6m20s
sauvegarde-29872535      Complete   1/1           3s         5m20s
sauvegarde-29872536      Complete   1/1           3s         4m20s
sauvegarde-vol-29872534  Complete   1/1           3s         6m20s
sauvegarde-vol-29872535  Complete   1/1           3s         5m20s
sauvegarde-vol-29872536  Complete   1/1           3s         4m20s
arij@master:~/lab6.1$ kubectl get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   A
GE
backup-pvc    Bound     pvc-bb6d26e5-dfd8-4fbf-bd52-43131f58773c   1Gi        RWO            local-path     <unset>                 1
1m
data          Bound     pvc-d747551f-e146-4ef4-bd75-424f7e2b605a   1Gi        RWO            local-path     <unset>                 3
h19m
data-db-0     Bound     pvc-c305b613-a5a7-40c5-8ee5-b608e9948a1c   1Gi        RWO            local-path     <unset>                 1
37m
data-erreur   Pending                                              0          RWO            local-path     <unset>                 3
h11m
arij@master:~/lab6.1$
```

3. Créer le pod "lecteur" pour vérifier les fichiers

```
arij@master:~/lab6.1$ cat <<'EOF' > lecteur.yaml
apiVersion: v1
kind: Pod
metadata:
  name: lecteur
spec:
  containers:
    - name: lecteur
      image: nginx:1.26-alpine
      volumeMounts:
        - name: backup
          mountPath: /backup
  volumes:
    - name: backup
      persistentVolumeClaim:
        claimName: backup-pvc
EOF
```

```
arij@master:~/lab6.1$ kubectl get jobs
NAME                     STATUS     COMPLETIONS   DURATION   AGE
calcul                   Complete   1/1           9s         99m
echec                    Failed     0/1           74m        74m
echec-onfailure          Failed     0/1           71m        71m
sauvegarde-29872541      Complete   1/1           3s         2m35s
sauvegarde-29872542      Complete   1/1           3s         95s
sauvegarde-29872543      Complete   1/1           3s         35s
sauvegarde-vol-29872541  Complete   1/1           3s         2m35s
sauvegarde-vol-29872542  Complete   1/1           3s         95s
sauvegarde-vol-29872543  Complete   1/1           3s         35s
```

```
arij@master:~/lab6.1$ kubectl apply -f lecteur.yaml
pod/lecteur created
arij@master:~/lab6.1$ kubectl wait --for=condition=Ready pod/lecteur --timeout=60s
pod/lecteur condition met
```

4. Lire les sauvegardes depuis le pod "lecteur"

```
arij@master:~/lab6.1$ kubectl exec lecteur -- ls -l /backup
total 40
-rw-r--r--    1 root     root            43 Oct 18 19:30 dump-20261018-193005.txt
-rw-r--r--    1 root     root            43 Oct 18 19:31 dump-20261018-193101.txt
-rw-r--r--    1 root     root            43 Oct 18 19:32 dump-20261018-193201.txt
-rw-r--r--    1 root     root            43 Oct 18 19:33 dump-20261018-193301.txt
-rw-r--r--    1 root     root            43 Oct 18 19:34 dump-20261018-193401.txt
-rw-r--r--    1 root     root            43 Oct 18 19:35 dump-20261018-193501.txt
-rw-r--r--    1 root     root            43 Oct 18 19:36 dump-20261018-193601.txt
-rw-r--r--    1 root     root            43 Oct 18 19:40 dump-20261018-194053.txt
-rw-r--r--    1 root     root            43 Oct 18 19:41 dump-20261018-194100.txt
-rw-r--r--    1 root     root            43 Oct 18 19:42 dump-20261018-194201.txt
```

5. lire le contenu du dernier fichier créé

```
arij@master:~/lab6.1$ kubectl exec lecteur -- sh -c 'cat /backup/$(ls /backup | tail -n 1)'
sauvegarde du Sun Oct 18 19:45:01 UTC 2026
arij@master:~/lab6.1$
```

Lab 6.2 : Patterns multi-conteneurs

Étape_1 : Préparer le namespace :

```
arij@master:~/lab6.1$ cd ~
mkdir lab6.2
cd lab6.2
kubectl config set-context --current --namespace=tp-k8s
Context "default" modified.
```

Étape_2 : Init containers :

1.Créer le fichier appli.yaml :

```
arij@master:~/lab6.2$ cat <<'EOF' > appli.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: appli
spec:
  replicas: 1
  selector:
    matchLabels:
      app: appli
  template:
    metadata:
      labels:
        app: appli
    spec:
      initContainers:
        - name: attendre-backend
          image: busybox:1.36
          command: ["sh", "-c", "until wget -q -T 2 -O /dev/null http://backend; do echo attente du backend; sleep 2; done; echo back
end disponible"]
        - name: preparer-contenu
          image: busybox:1.36
          command: ["sh", "-c", "echo 'Page generee par un init container' > /html/index.html"]
          volumeMounts:
            - name: html
              mountPath: /html
      containers:
        - name: appli
          image: nginx:1.26-alpine
          volumeMounts:
            - name: html
              mountPath: /usr/share/nginx/html
```

```
arij@master:~/lab6.2$ kubectl apply -f appli.yaml
deployment.apps/appli created
arij@master:~/lab6.2$ sleep 15
arij@master:~/lab6.2$ kubectl get pods
NAME                              READY   STATUS      RESTARTS      AGE
api-7779574f48-94phk              1/1     Running     0             6h32m
api-7779574f48-rp4rb              1/1     Running     0             6h32m
api-7779574f48-t86nd              1/1     Running     0             6h32m
app                               1/1     Running     0             3h32m
appli-6896f9c74f-wbr7n            0/1     Init:0/2    0             58s
broken                            1/1     Running     1 (8h ago)    6d9h
calcul-jkpzb                      0/1     Completed   0             108m
db-0                              1/1     Running     0             124m
demo-7b48f66b47-cdp56             1/1     Running     1 (8h ago)    6d9h
echec-2s76p                       0/1     Error       0             83m
echec-cmmxc                       0/1     Error       0             83m
echec-nhqng                       0/1     Error       0             83m
lecteur                           1/1     Running     0             11m
podinfo-f6698f896-j5dz8           1/1     Running     0             7h54m
podinfo-f6698f896-x7q4d           1/1     Running     0             7h54m
podinfo-f6698f896-zvgt2           1/1     Running     0             7h54m
sauvegarde-29872550-klncb         0/1     Completed   0             2m44s
sauvegarde-29872551-428zj         0/1     Completed   0             104s
sauvegarde-29872552-lzfbt         0/1     Completed   0             44s
sauvegarde-vol-29872550-8f4dl     0/1     Completed   0             2m44s
sauvegarde-vol-29872551-hj427     0/1     Completed   0             104s
sauvegarde-vol-29872552-w859h     0/1     Completed   0             44s
temp                              1/1     Running     0             4h1m
web-79b7768d68-gv5v9              1/1     Running     0             6h54m
web-79b7768d68-v9bsv              1/1     Running     0             6h54m
arij@master:~/lab6.2$ kubectl logs deploy/appli -c attendre-backend --tail=3
attente du backend
wget: bad address 'backend'
```

2. Créer le backend dans un autre terminal (ou en arrière-plan)

*Terminal 2

```
arij@master:~/lab6.1$ kubectl get pods -w
NAME                          READY   STATUS      RESTARTS      AGE
api-7779574f48-94phk          1/1     Running     0             6h33m
api-7779574f48-rp4rb          1/1     Running     0             6h33m
api-7779574f48-t86nd          1/1     Running     0             6h33m
app                           1/1     Running     0             3h33m
appli-6896f9c74f-wbr7n        0/1     Init:0/2    0             2m6s
broken                        1/1     Running     1 (8h ago)    6d9h
calcul-jkpzb                  0/1     Completed   0             110m
db-0                          1/1     Running     0             125m
demo-7b48f66b47-cdp56         1/1     Running     1 (8h ago)    6d9h
echec-2s76p                   0/1     Error       0             84m
echec-cmmxc                   0/1     Error       0             84m
echec-nhqng                   0/1     Error       0             85m
lecteur                       1/1     Running     0             12m
podinfo-f6698f896-j5dz8       1/1     Running     0             7h55m
podinfo-f6698f896-x7q4d       1/1     Running     0             7h55m
podinfo-f6698f896-zvgt2       1/1     Running     0             7h55m
sauvegarde-29872551-428zj     0/1     Completed   0             2m52s
sauvegarde-29872552-lzfbt     0/1     Completed   0             112s
sauvegarde-29872553-gfrvs     0/1     Completed   0             52s
sauvegarde-vol-29872551-hj427 0/1     Completed   0             2m52s
sauvegarde-vol-29872552-w859h 0/1     Completed   0             112s
sauvegarde-vol-29872553-mmjzc 0/1     Completed   0             52s
temp                          1/1     Running     0             4h2m
web-79b7768d68-gv5v9          1/1     Running     0             6h55m
web-79b7768d68-v9bsv          1/1     Running     0             6h55m
```

*Terminal 1 :

```
arij@master:~/lab6.2$ cat <<'EOF' > backend.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: nginx:1.26-alpine
---
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend
  ports:
    - port: 80
EOF
arij@master:~/lab6.2$ kubectl apply -f backend.yaml
deployment.apps/backend created
service/backend created
arij@master:~/lab6.2$
```

*Résultat sur Terminal 2

```
sauvegarde-vol-29872552-w859h   0/1     Completed           0             3m3s
backend-6f69cbdfd7-qbjvv        0/1     Pending             0             1s
backend-6f69cbdfd7-qbjvv        0/1     Pending             0             1s
backend-6f69cbdfd7-qbjvv        0/1     ContainerCreating   0             1s
backend-6f69cbdfd7-qbjvv        1/1     Running             0             1s
appli-6896f9c74f-wbr7n          0/1     Init:1/2            0             3m38s
appli-6896f9c74f-wbr7n          0/1     Init:1/2            0             3m38s
appli-6896f9c74f-wbr7n          0/1     PodInitializing     0             3m39s
appli-6896f9c74f-wbr7n          1/1     Running             0             3m39s
```

3. Vérifier le résultat

```
arij@master:~/lab6.2$ kubectl logs deploy/appli -c attendre-backend --tail=2
attente du backend
backend disponible
arij@master:~/lab6.2$ kubectl exec deploy/appli -c appli -- wget -qO- http://localhost
Page generee par un init container
arij@master:~/lab6.2$
```

Étape_3 : Diagnostiquer un init container en échec :

1. Créer le fichier init-erreur.yaml

```
arij@master:~/lab6.2$ cat <<'EOF' > init-erreur.yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-erreur
spec:
  initContainers:
    - name: init-casse
      image: busybox:1.36
      command: ["sh", "-c", "echo preparation en cours; exit 1"]
  containers:
    - name: app
      image: nginx:1.26-alpine
EOF
```

2. Diagnostiquer l'échec :

```
arij@master:~/lab6.2$ sleep 30
arij@master:~/lab6.2$ kubectl get pod init-erreur
NAME          READY   STATUS       RESTARTS      AGE
init-erreur   0/1     Init:Error   3 (28s ago)   44s
arij@master:~/lab6.2$ kubectl logs init-erreur -c init-casse
preparation en cours
arij@master:~/lab6.2$ kubectl describe pod init-erreur | grep -A8 "Init Containers"
Init Containers:
  init-casse:
    Container ID:  containerd://b5a46c04d43dc9ac20d07fed1c6ea3b7c6c3c359365a9c31385496c7f6de45b3
    Image:         busybox:1.36
    Image ID:      docker.io/library/busybox@sha256:73aaf090f3d85aa34ee199857f03fa3a95c8ede2ffd4cc2cdb5b94e566b11662
    Port:          <none>
    Host Port:     <none>
    Command:
      sh
```

Étape_4 : Sidecar, volume partagé et réseau partagé :

1.Creer les logs dans le fichier

```
arij@master:~/lab6.2$ cat <<'EOF' > app-logs.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-logs
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date >> /logs/app.log; sleep 5; done"]
      volumeMounts:
        - name: logs
          mountPath: /logs
    - name: lecteur-logs
      image: busybox:1.36
      command: ["sh", "-c", "tail -F /logs/app.log"]
      volumeMounts:
        - name: logs
          mountPath: /logs
  volumes:
    - name: logs
      emptyDir: {}
EOF
arij@master:~/lab6.2$ kubectl apply -f app-logs.yaml
pod/app-logs created
arij@master:~/lab6.2$ kubectl wait --for=condition=Ready pod/app-logs --timeout=60s
pod/app-logs condition met
```

2. Vérification

```
arij@master:~/lab6.2$ sleep 15
arij@master:~/lab6.2$ kubectl get pod app-logs
NAME       READY   STATUS    RESTARTS   AGE
app-logs   2/2     Running   0          61s
arij@master:~/lab6.2$ kubectl logs app-logs -c app
arij@master:~/lab6.2$ kubectl logs app-logs -c lecteur-logs --tail=3
Sun Oct 18 20:03:39 UTC 2026
Sun Oct 18 20:03:44 UTC 2026
Sun Oct 18 20:03:49 UTC 2026
```

3. Le réseau partagé (localhost)

```
arij@master:~/lab6.2$ cat <<'EOF' > reseau-partage.yaml
apiVersion: v1
kind: Pod
metadata:
  name: reseau-partage
spec:
  containers:
    - name: web
      image: nginx:1.26-alpine
    - name: testeur
      image: busybox:1.36
      command: ["sh", "-c", "while true; do wget -qO- http://localhost | grep -o '<title>.*</title>'; sleep 10; done"]
EOF
arij@master:~/lab6.2$ kubectl apply -f reseau-partage.yaml
pod/reseau-partage created
arij@master:~/lab6.2$ kubectl wait --for=condition=Ready pod/reseau-partage --timeout=60s
pod/reseau-partage condition met
```

4.Vérification :

```
arij@master:~/lab6.2$ sleep 15
arij@master:~/lab6.2$ kubectl logs reseau-partage -c testeur --tail=2
<title>Welcome to nginx!</title>
<title>Welcome to nginx!</title>
```

Étape_5 : Ambassador :

1. Créer le fichier ambassador.yaml :

```
arij@master:~/lab6.2$ cat <<'EOF' > ambassador.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: ambassador-conf
data:
  default.conf: |
    server {
      listen 8080;
      location / {
        proxy_pass http://backend:80;
      }
    }
---
apiVersion: v1
kind: Pod
metadata:
  name: ambassador
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do wget -qO- http://localhost:8080 2>&1 | grep -Eo '<title>.*</title>|wget: .*'; sleep 5; do
ne"]
    - name: proxy
      image: nginx:1.26-alpine
      volumeMounts:
        - name: conf
          mountPath: /etc/nginx/conf.d/default.conf
          subPath: default.conf
  volumes:
    - name: conf
      configMap:
```

```
arij@master:~/lab6.2$ kubectl apply -f ambassador.yaml
configmap/ambassador-conf created
pod/ambassador created
arij@master:~/lab6.2$ kubectl wait --for=condition=Ready pod/ambassador --timeout=60s
pod/ambassador condition met
```

2. Vérification

```
arij@master:~/lab6.2$ kubectl logs ambassador -c app --tail=3
<title>Welcome to nginx!</title>
<title>Welcome to nginx!</title>
<title>Welcome to nginx!</title>
```

3. Couper le backend et observer

```
arij@master:~/lab6.2$ kubectl scale deployment backend --replicas=0
deployment.apps/backend scaled
arij@master:~/lab6.2$ sleep 15
arij@master:~/lab6.2$ kubectl logs ambassador -c app --tail=3
<title>Welcome to nginx!</title>
<title>Welcome to nginx!</title>
<title>Welcome to nginx!</title>
```

4. Rétablir le backend

```
arij@master:~/lab6.2$ kubectl scale deployment backend --replicas=1
deployment.apps/backend scaled
arij@master:~/lab6.2$ kubectl rollout status deployment/backend
deployment "backend" successfully rolled out
arij@master:~/lab6.2$ sleep 15
arij@master:~/lab6.2$ kubectl logs ambassador -c app --tail=3
<title>Welcome to nginx!</title>
<title>Welcome to nginx!</title>
<title>Welcome to nginx!</title>
```